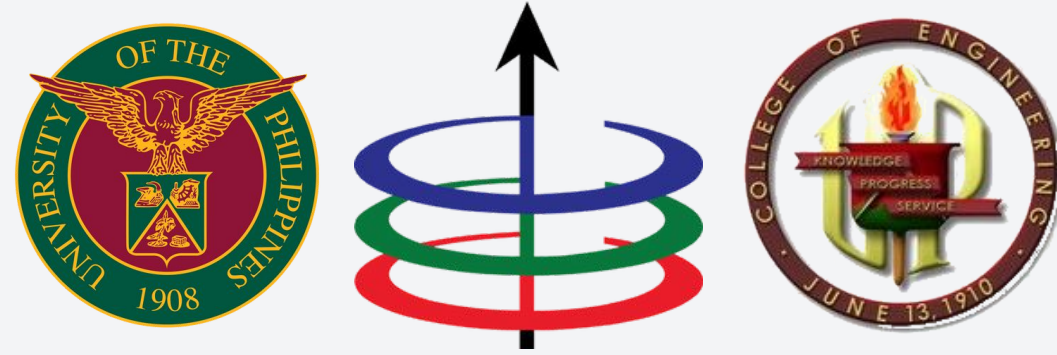




Lecture 5

Non-linear Data Structures

EEE 121 2s2526

Steven S. Sison

Before everything else...

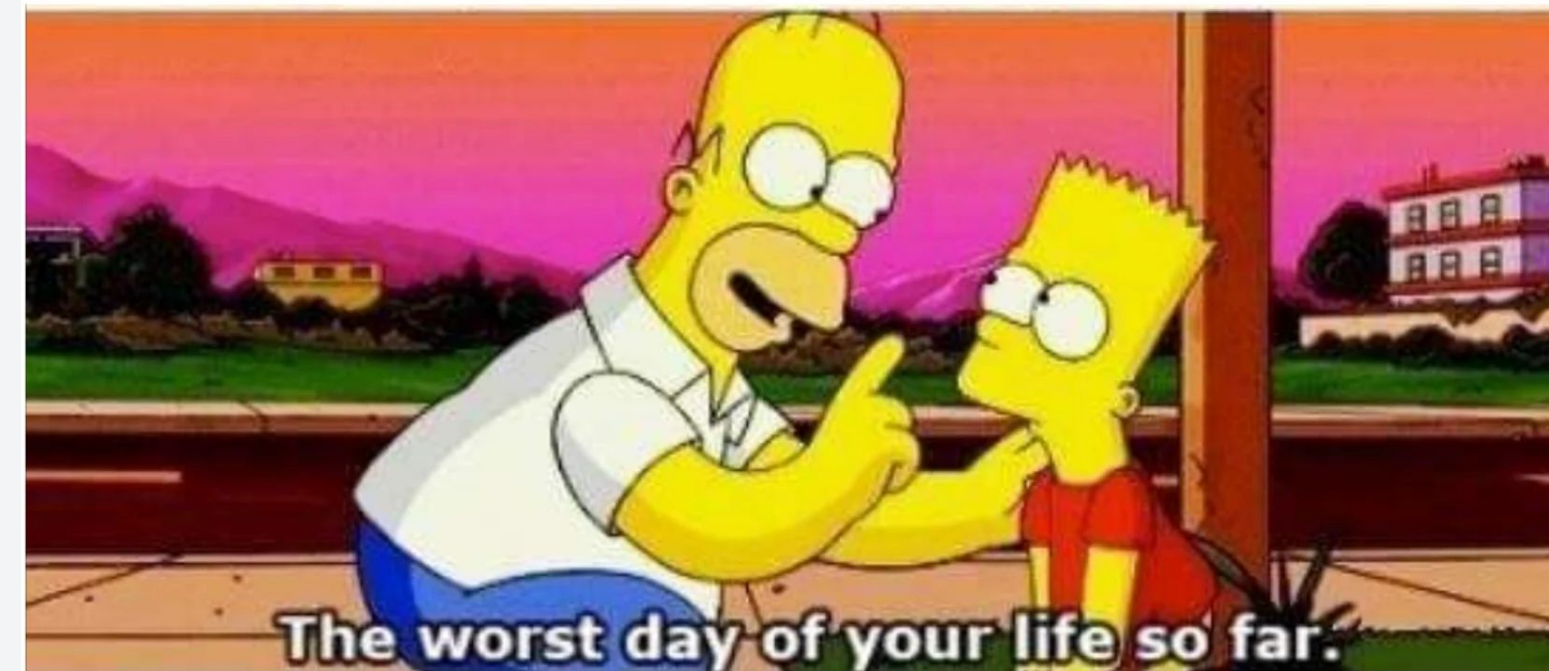
We'll have our midterms on **Monday (March 23)**

- Where: TBA, UP EEEl
- When:
 - 9-11AM (Regular exam)
 - 8-10AM (Early exam)
 - 10AM-12NN (Late exam)
- Topics: Lecture 0 to Lecture 5

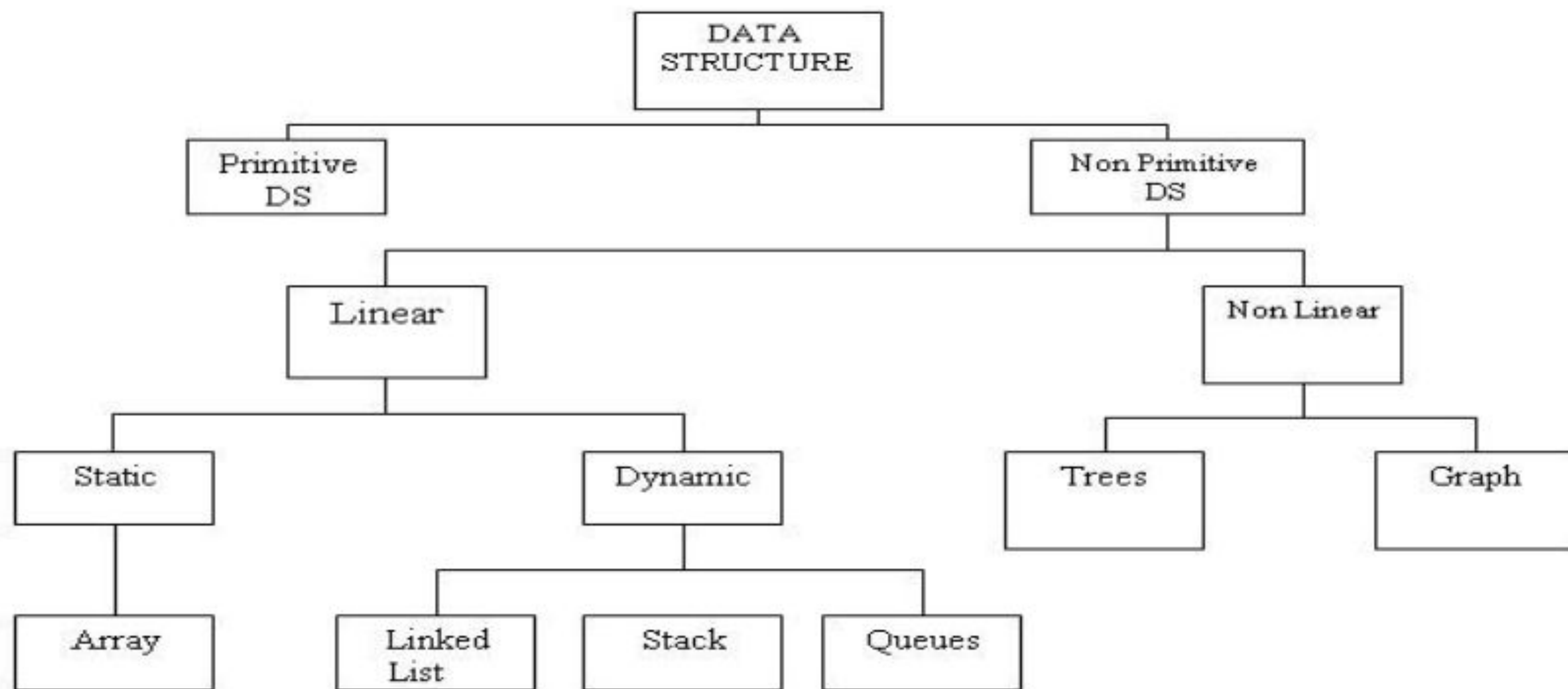
Me, after the data structures exam:

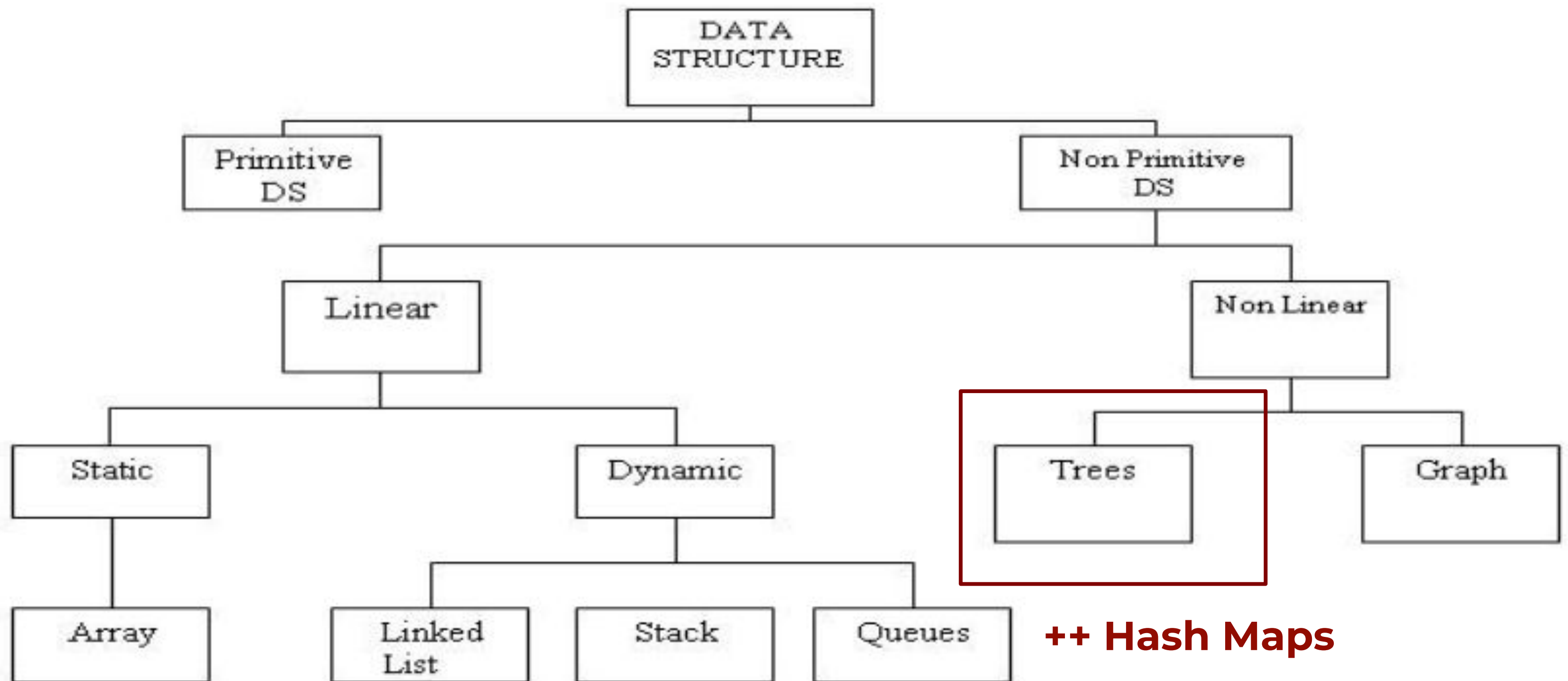


The academic institution I study at:



Ready?





- Trees and Heap
- Hash Map

- **Trees and Heap**
- Hash Map

Arrays and Linked Lists are good, but may be slow

Certain algorithms work better with linear data structures, but some may not and can slow it down.

Non-linear Data Structures

- Some algorithms work better with non-linear data structures.
 - File structures are represented by trees to show hierarchy.
 - Road networks are modeled by graphs to show interconnections.
- Some known non-linear data structures:
 - Trees
 - Maps
 - Sets and Graphs

Arrays and Linked Lists are good, but may be slow

Certain algorithms work better with linear data structures, but some may not and can slow it down.

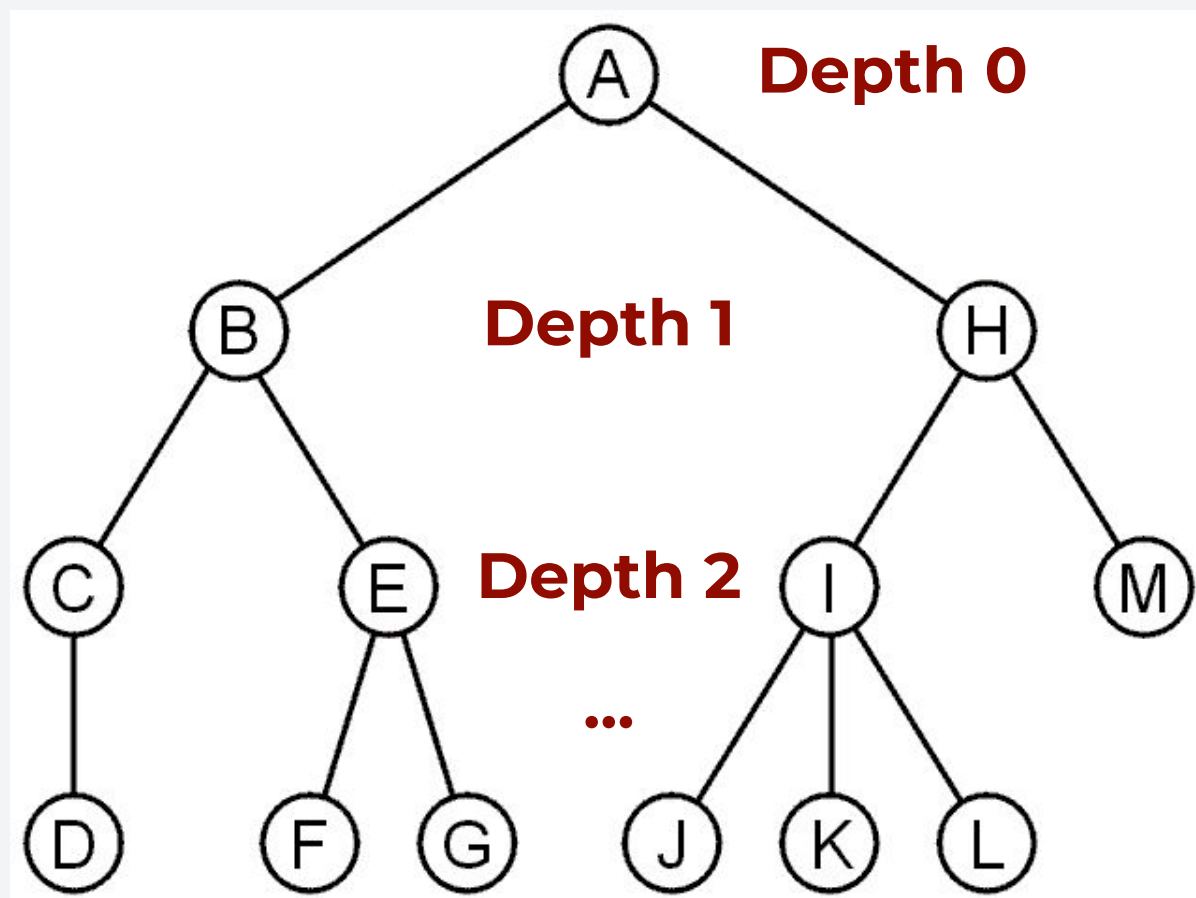
Non-linear Data Structures

- Some algorithms work better with non-linear data structures.
 - File structures are represented by trees to show hierarchy.
 - Road networks are modeled by graphs to show interconnections.
- Some known non-linear data structures:
 - Trees
 - Maps
 - Sets and Graphs

The Tree Data Structure

Similar to linked lists, a (rooted) tree stores information in nodes

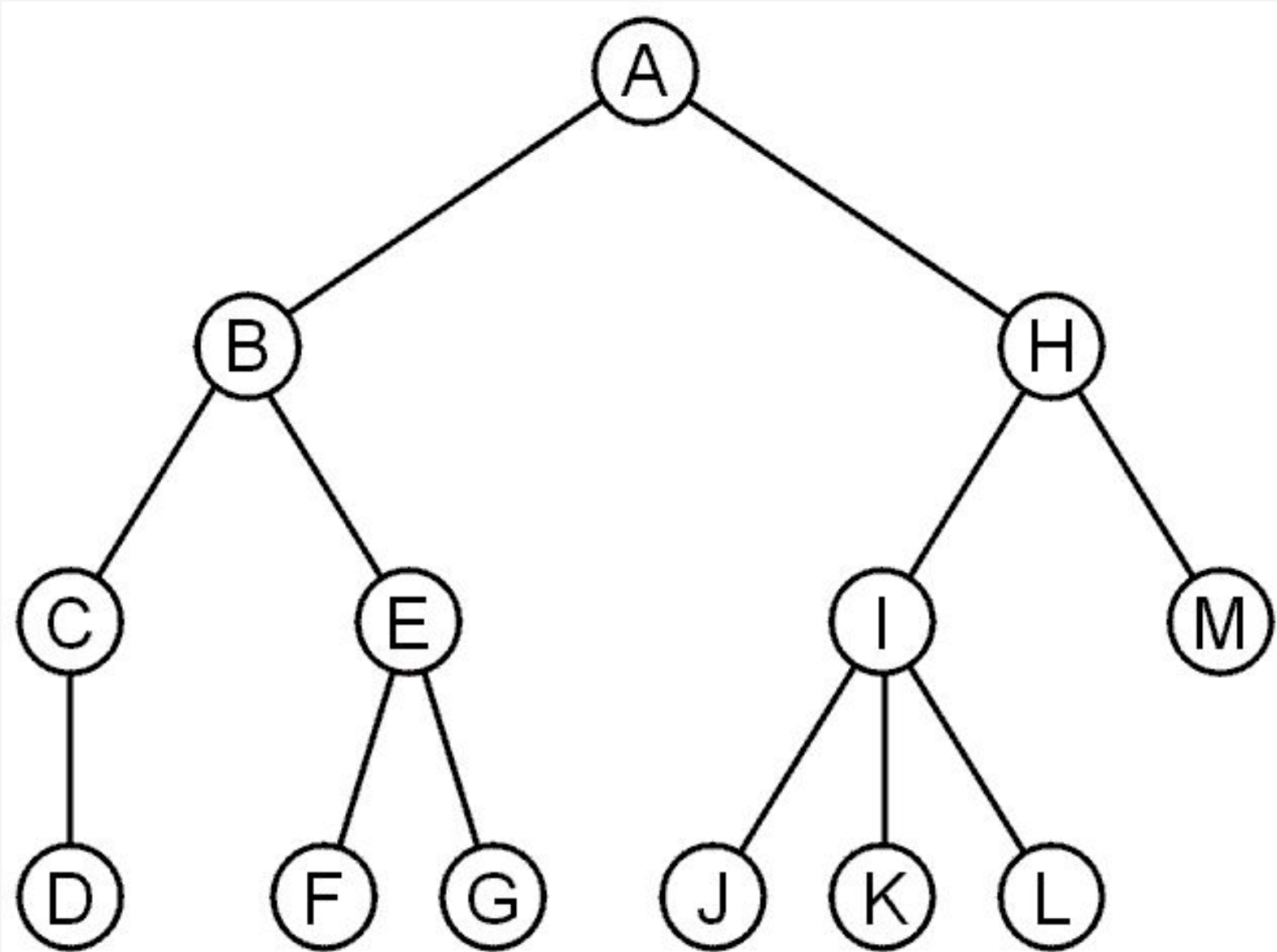
- There is a first node called **root (Node A)**
- Each node has a variable number of references to successors
- Each node, other than the root node, has exactly 1 node pointing to it



- Links connecting nodes are **unidirectional** (going from top to bottom)
- There is only **one possible path** from root node to any other nodes in the tree
- Length of path from root to a certain node is the **depth of a node**

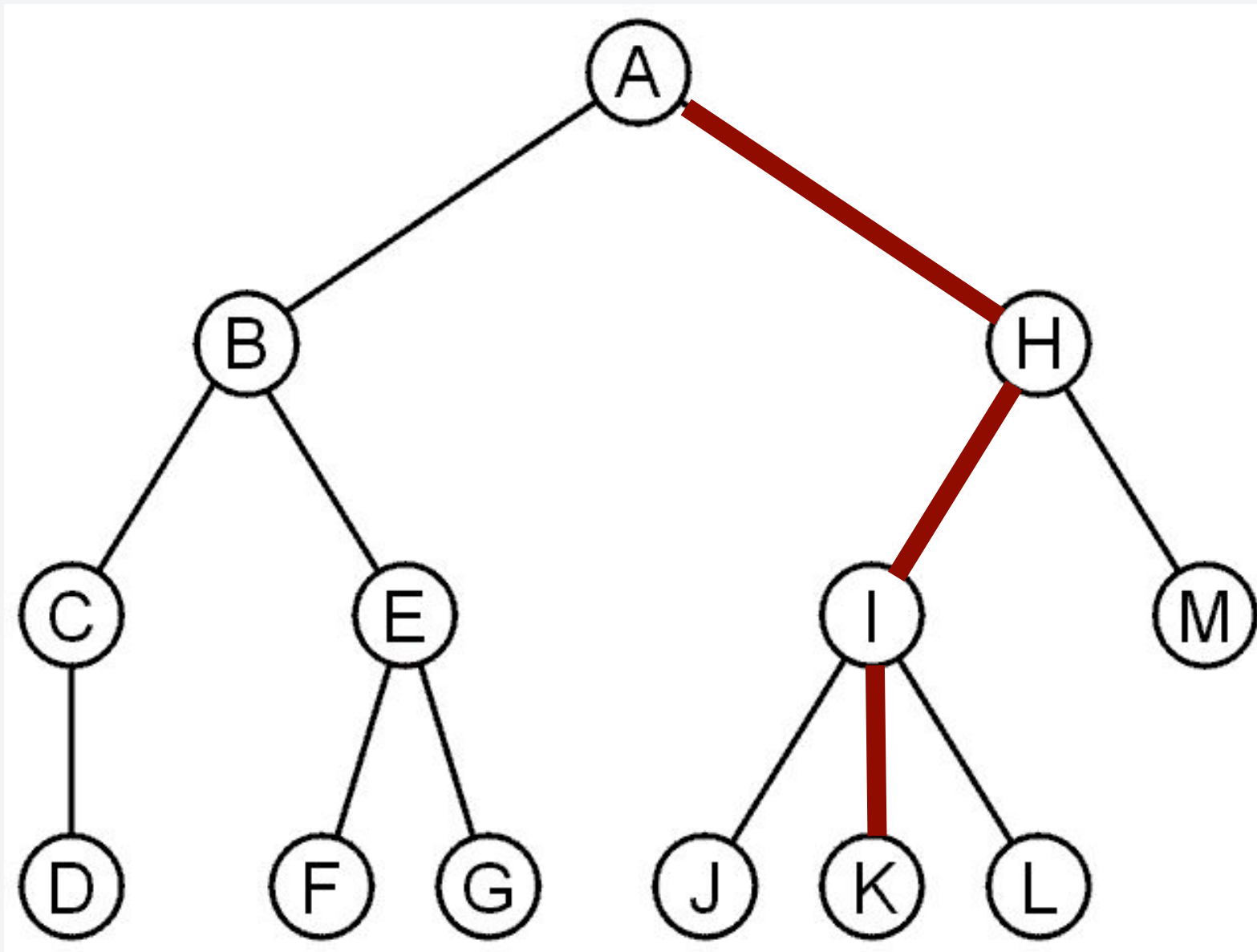
The Tree Data Structure

What is the path from A to K?



The Tree Data Structure

What is the path from A to K?

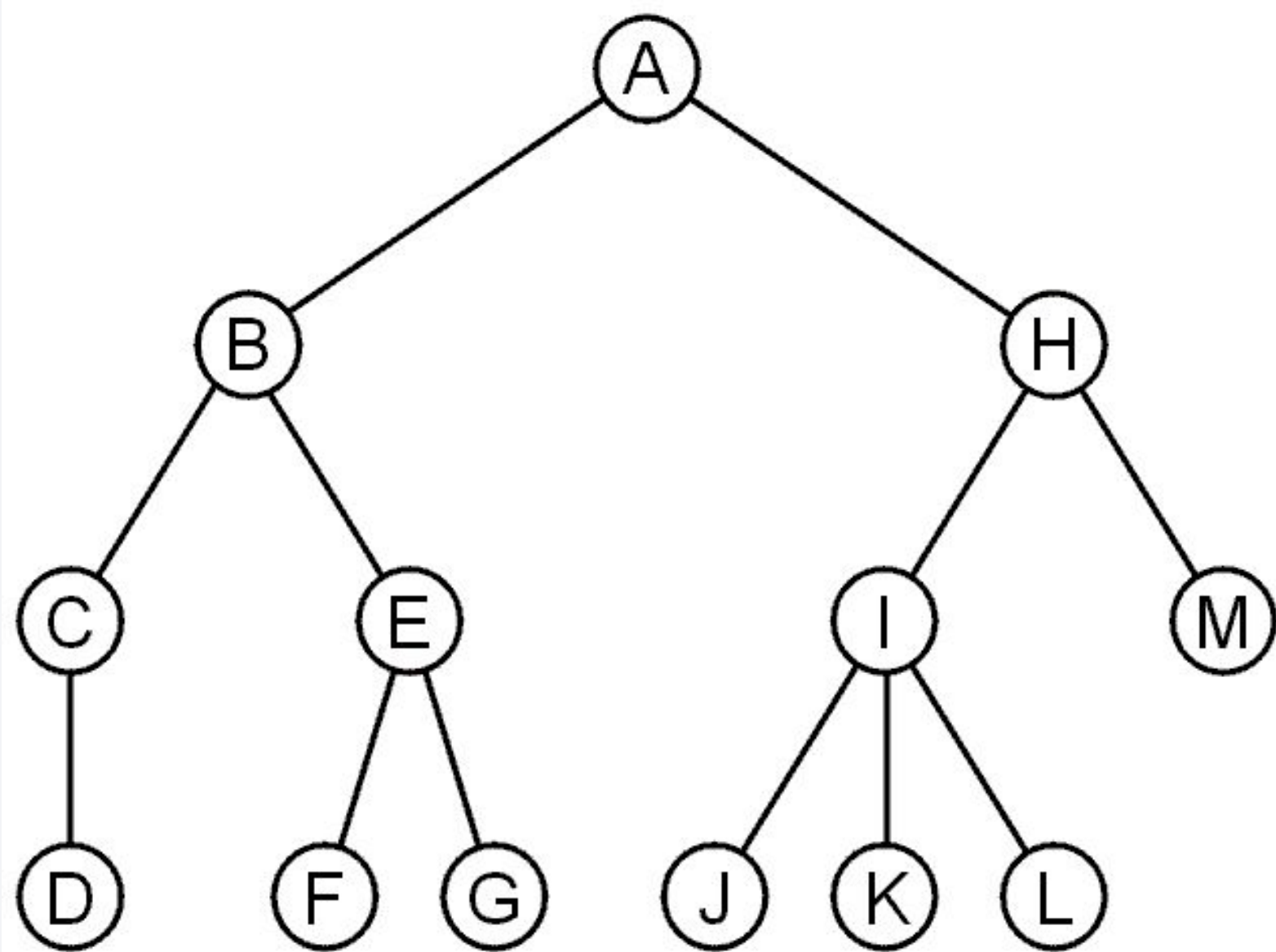


A -> H -> I -> K

Note. This is just an example to show you which nodes are connected. Graph traversals are often more complicated.

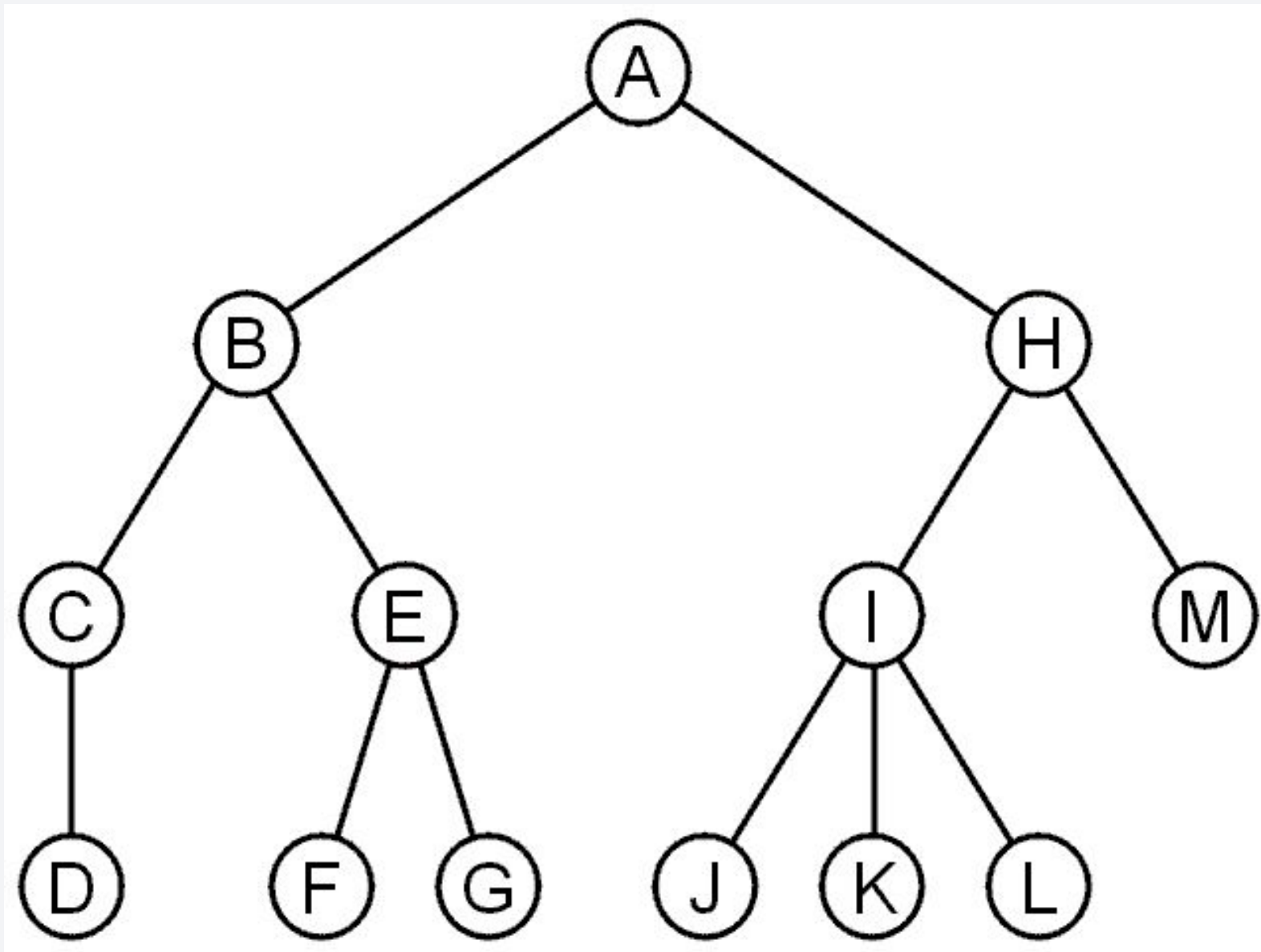
The Tree Data Structure

What is the path from I to E?



The Tree Data Structure

What is the path from I to E?

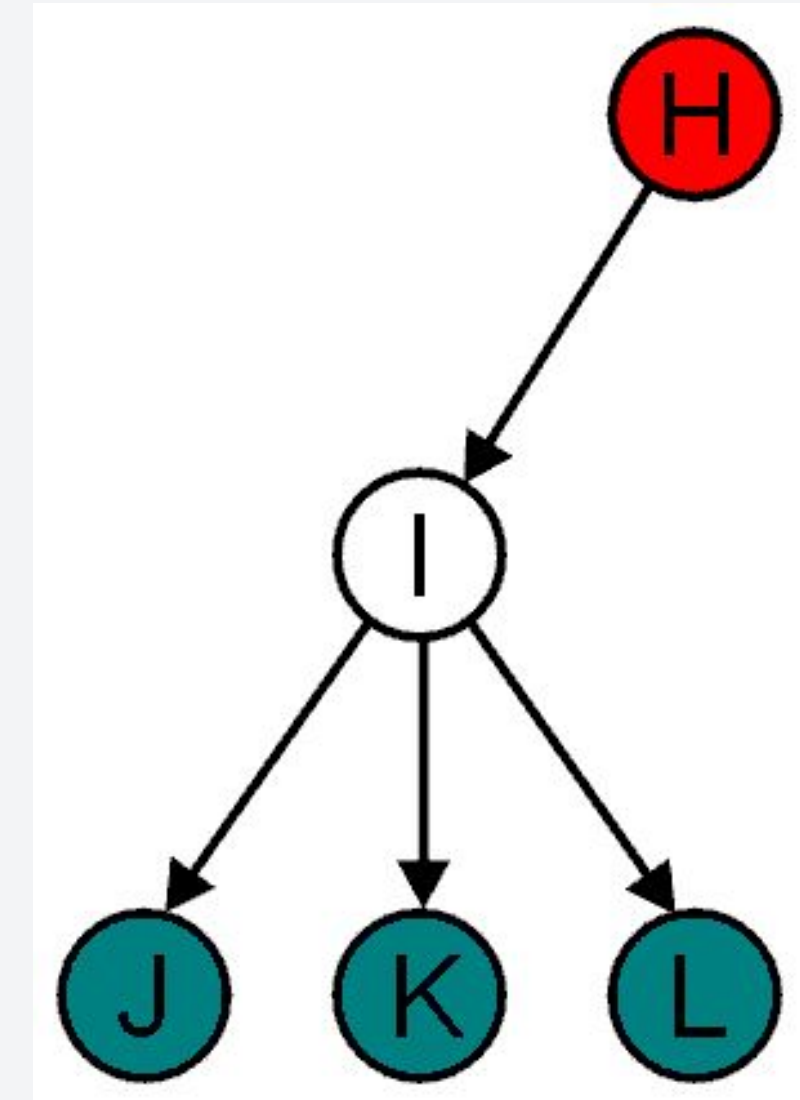


NONE!

A tree is unidirectional. Maybe in the future with graphs this is possible :D

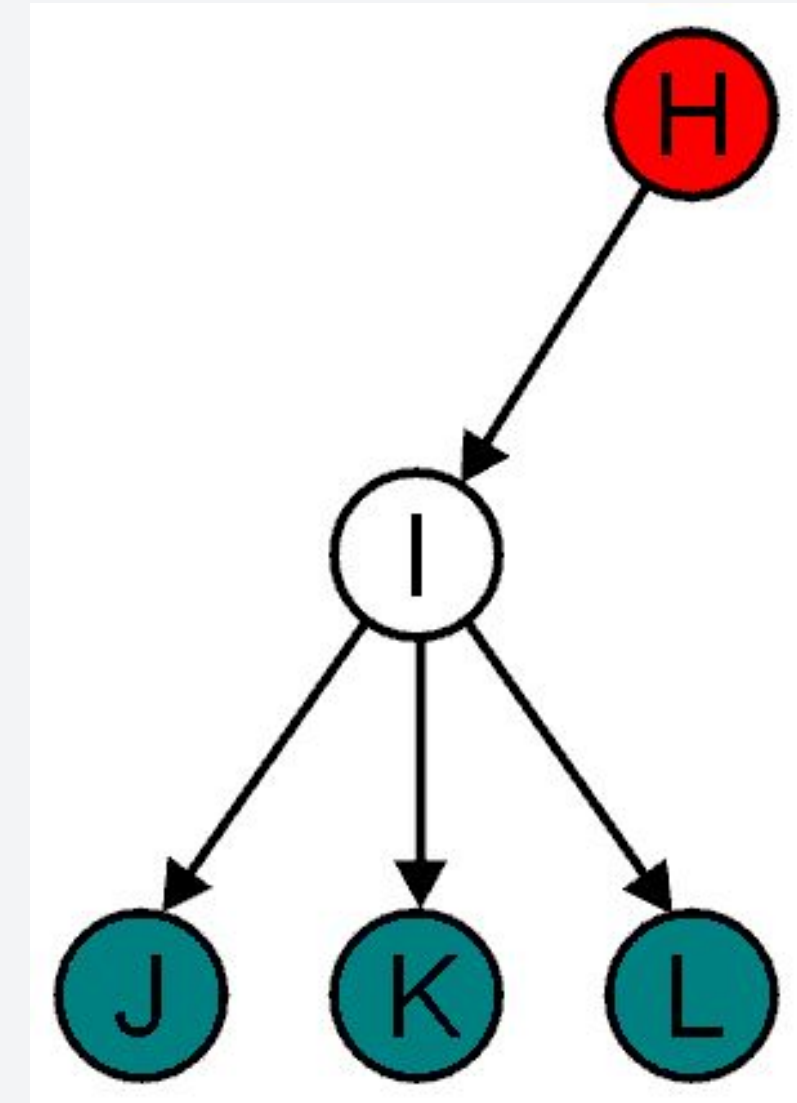
Tree Terminologies

- Each node can have zero or more child nodes called **children** (*I has three; J,K,L has zero*)
- The topmost part of the tree is called the **root**.
- For each node (except the root), there is **one parent node**.
 - Nodes having the same parents are called **siblings**. (*J,K,L are siblings*)
- The **degree** of a node is defined as the number of its children. ($\text{deg}(I) = 3$)
- The **height** of a tree refers to the maximum depth for all nodes in the tree (excluding the root).



Tree Terminologies

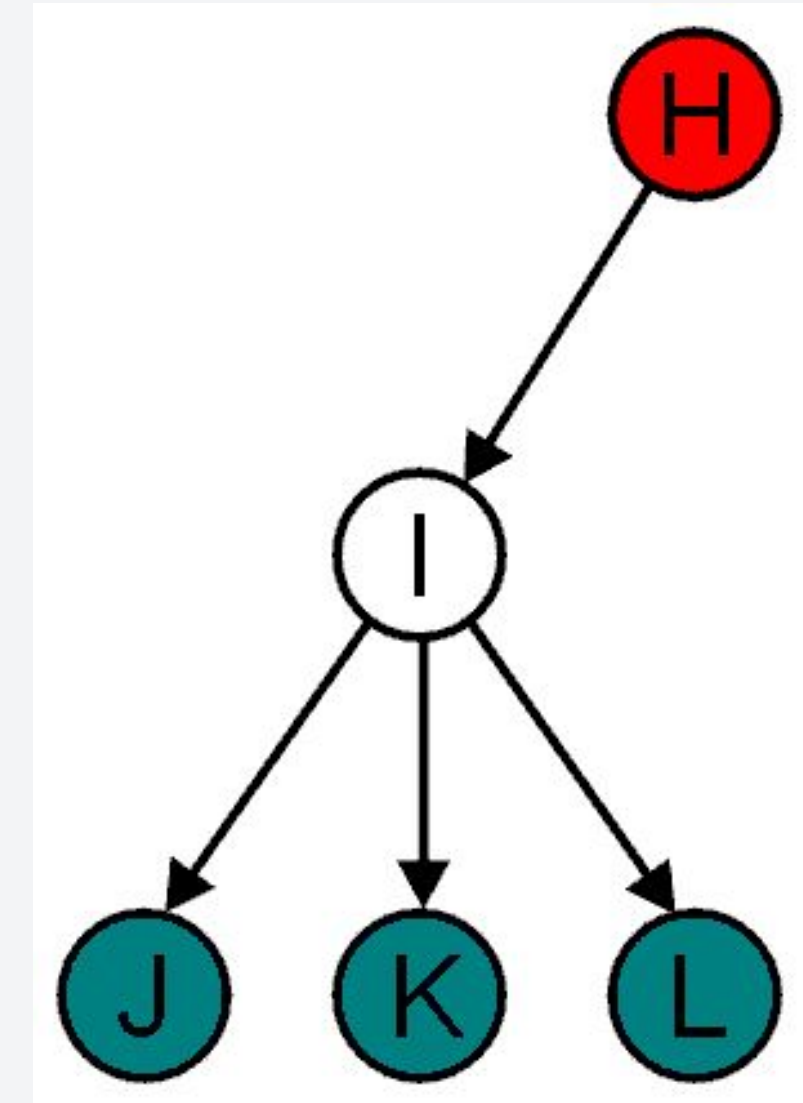
- Each node can have zero or more child nodes called **children** (*I has three; J,K,L has zero*)
- The topmost part of the tree is called the **root**.
- For each node (except the root), there is **one parent node**.
 - Nodes having the same parents are called **siblings**. (*J,K,L are siblings*)
- The **degree** of a node is defined as the number of its children. ($\text{deg}(I) = 3$)
- The **height** of a tree refers to the maximum depth for all nodes in the tree (excluding the root).



What is the root node?

Tree Terminologies

- Each node can have zero or more child nodes called **children** (*I has three; J,K,L has zero*)
- The topmost part of the tree is called the **root**.
- For each node (except the root), there is **one parent node**.
 - Nodes having the same parents are called **siblings**. (*J,K,L are siblings*)
- The **degree** of a node is defined as the number of its children. ($\text{deg}(I) = 3$)
- The **height** of a tree refers to the maximum depth for all nodes in the tree (excluding the root).

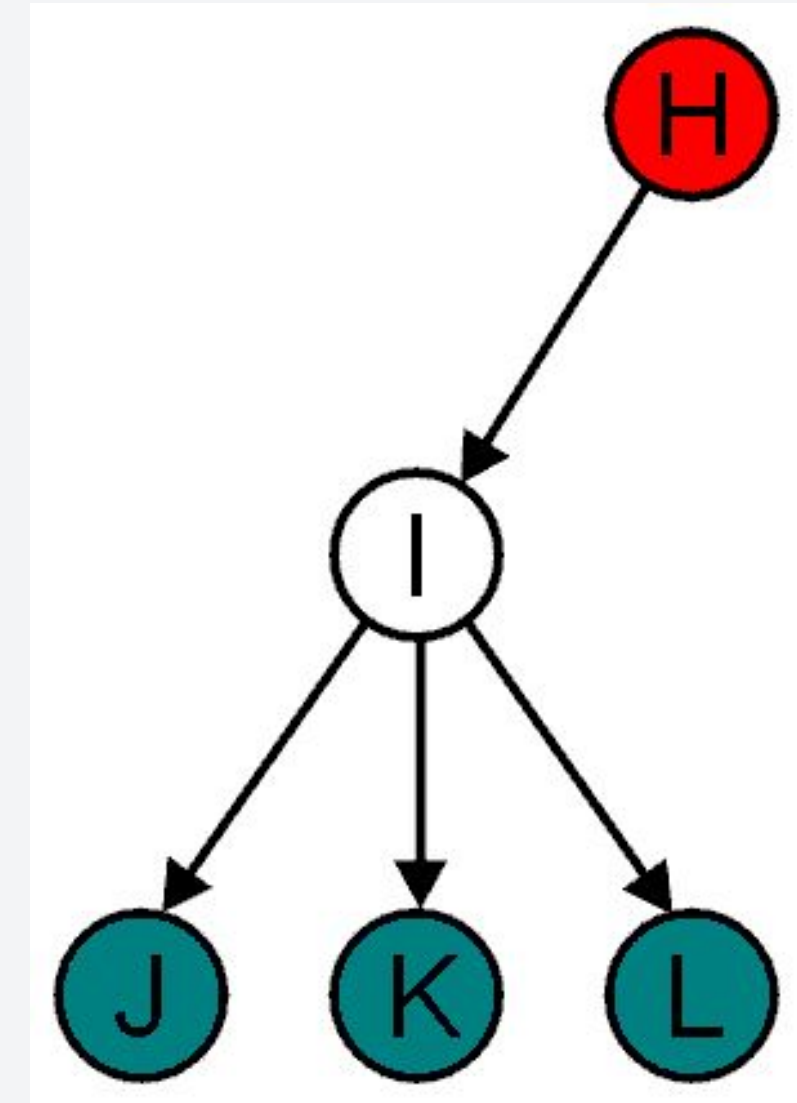


What is the root node?

H

Tree Terminologies

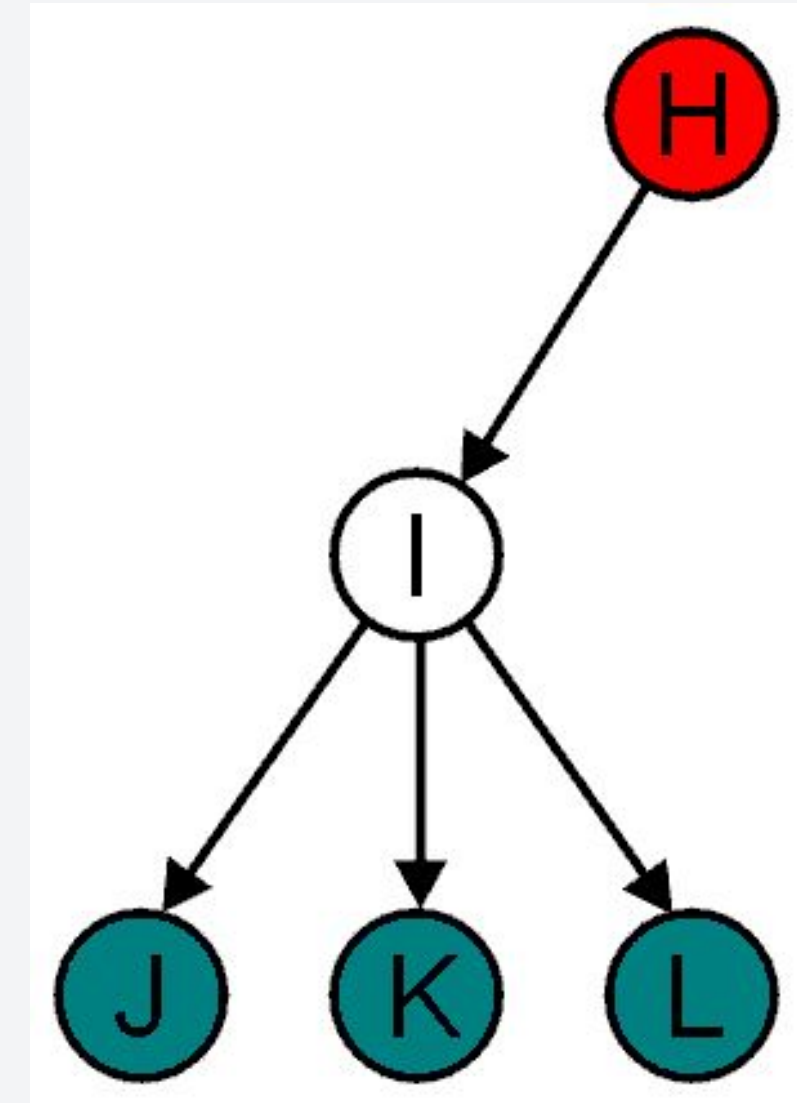
- Each node can have zero or more child nodes called **children** (*I has three; J,K,L has zero*)
- The topmost part of the tree is called the **root**.
- For each node (except the root), there is **one parent node**.
 - Nodes having the same parents are called **siblings**. (*J,K,L are siblings*)
- The **degree** of a node is defined as the number of its children. ($\text{deg}(I) = 3$)
- The **height** of a tree refers to the maximum depth for all nodes in the tree (excluding the root).



**How many children
does I Have?**

Tree Terminologies

- Each node can have zero or more child nodes called **children** (*I has three; J,K,L has zero*)
- The topmost part of the tree is called the **root**.
- For each node (except the root), there is **one parent node**.
 - Nodes having the same parents are called **siblings**. (*J,K,L are siblings*)
- The **degree** of a node is defined as the number of its children. ($\text{deg}(I) = 3$)
- The **height** of a tree refers to the maximum depth for all nodes in the tree (excluding the root).

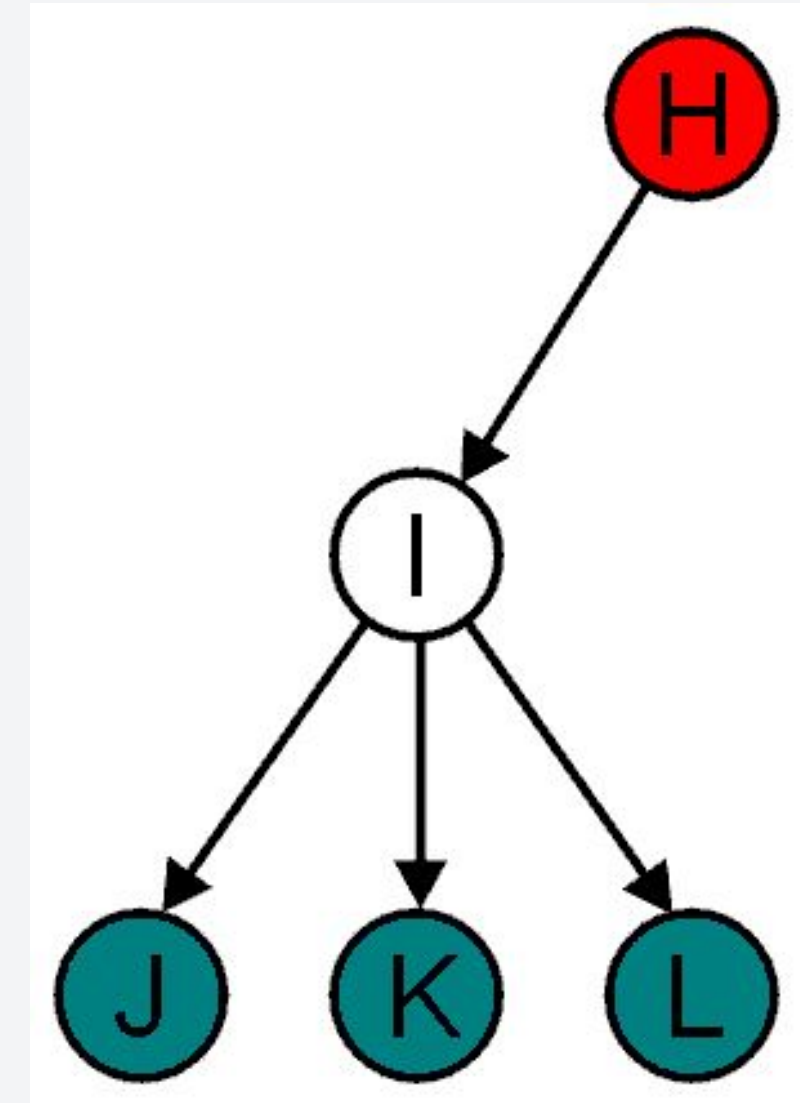


**How many children
does I Have?**

3

Tree Terminologies

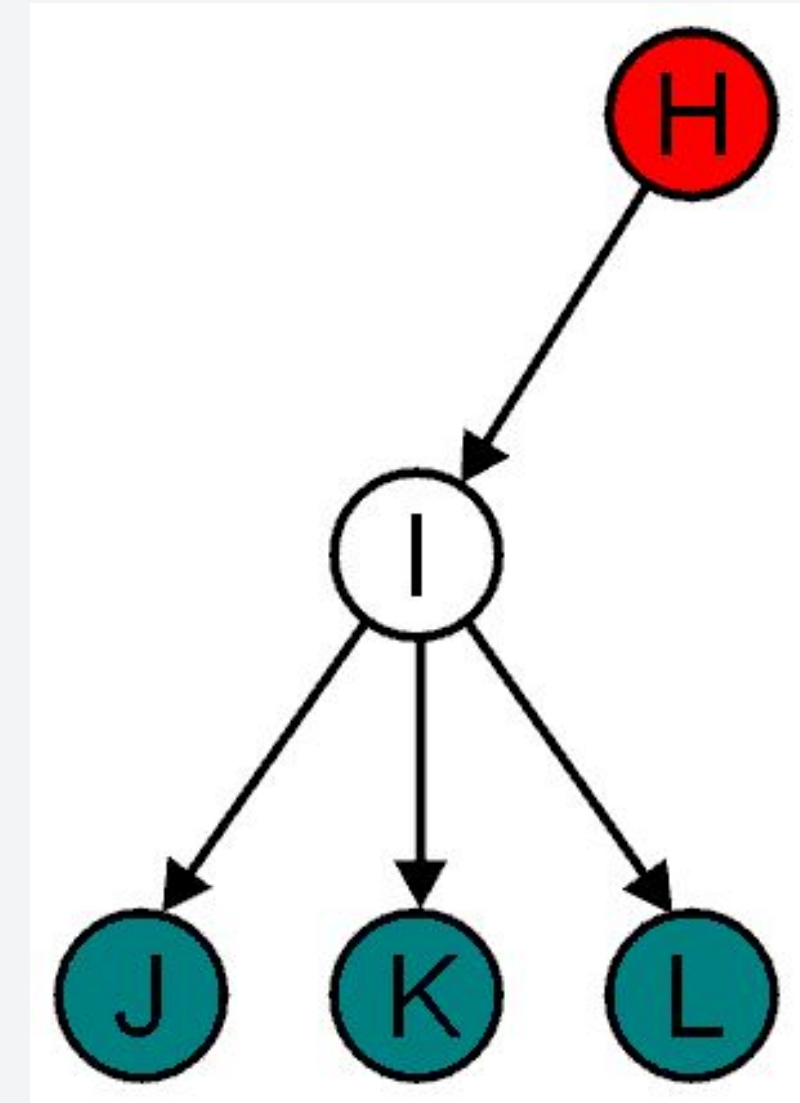
- Each node can have zero or more child nodes called **children** (*I has three; J,K,L has zero*)
- The topmost part of the tree is called the **root**.
- For each node (except the root), there is **one parent node**.
 - Nodes having the same parents are called **siblings**. (*J,K,L are siblings*)
- The **degree** of a node is defined as the number of its children. ($\text{deg}(I) = 3$)
- The **height** of a tree refers to the maximum depth for all nodes in the tree (excluding the root).



What is the height of the tree?

Tree Terminologies

- Each node can have zero or more child nodes called **children** (*I has three; J,K,L has zero*)
- The topmost part of the tree is called the **root**.
- For each node (except the root), there is **one parent node**.
 - Nodes having the same parents are called **siblings**. (*J,K,L are siblings*)
- The **degree** of a node is defined as the number of its children. ($\text{deg}(I) = 3$)
- The **height** of a tree refers to the maximum depth for all nodes in the tree (excluding the root).

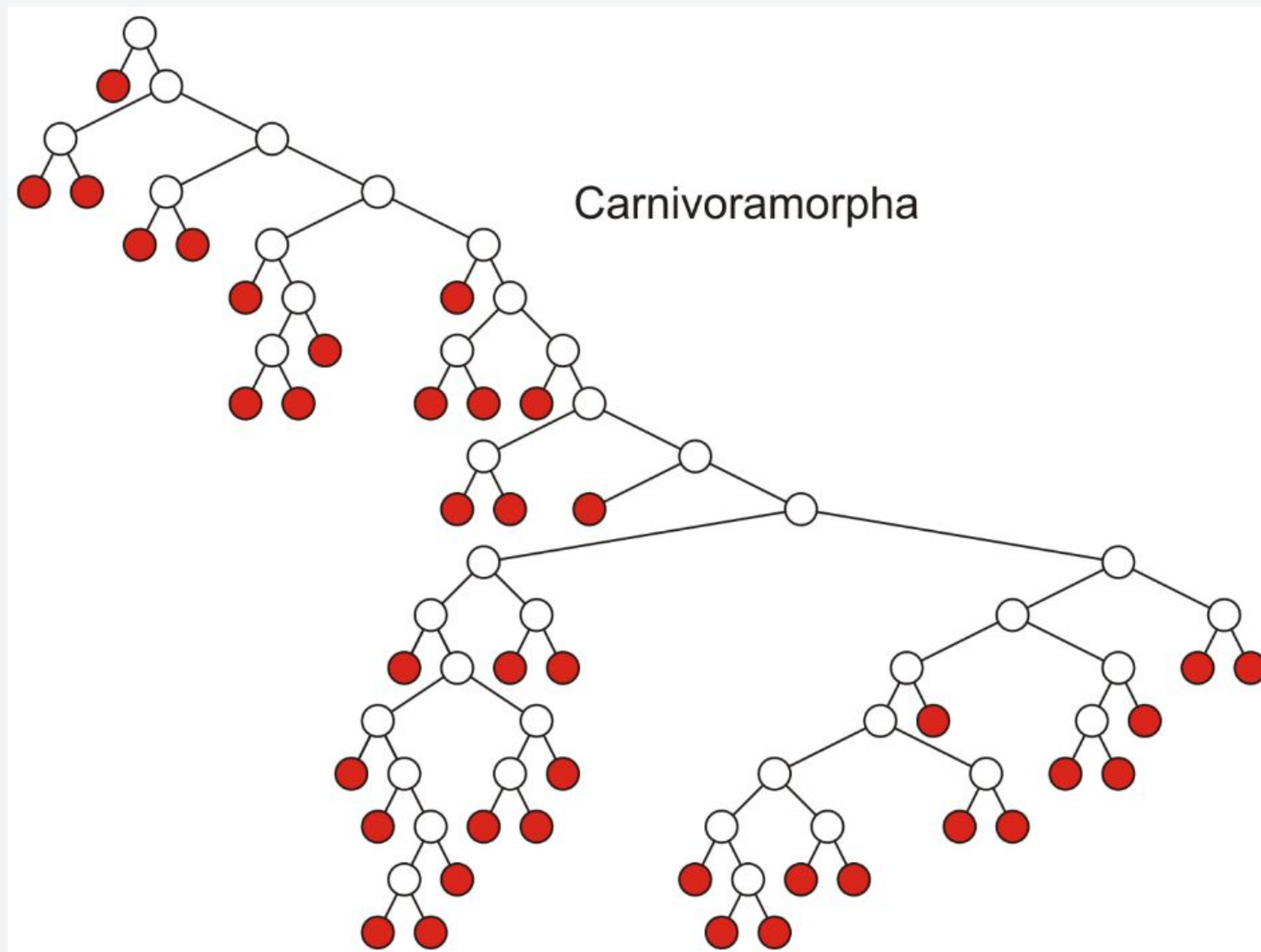


What is the height of the tree?

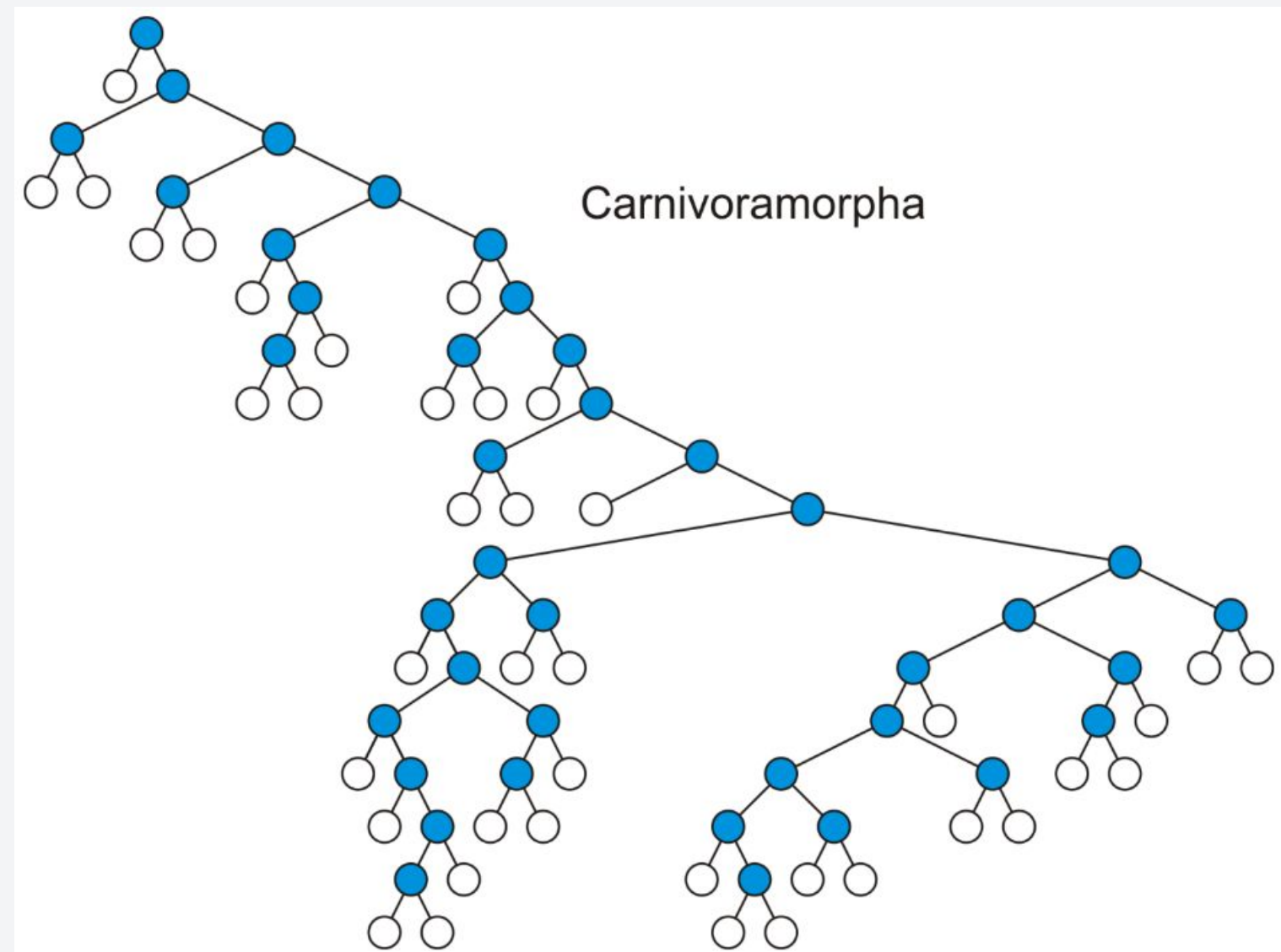
2

Tree Terminologies

- Nodes with degree zero are called **leaf nodes**.
- All other nodes are called **internal nodes**.



Wesley-Hunt, G. D.; Flynn, J. J. "Phylogeny of the Carnivora: basal relationships among the Carnivoramorpha, and assessment of the position of 'Miacoidea'"

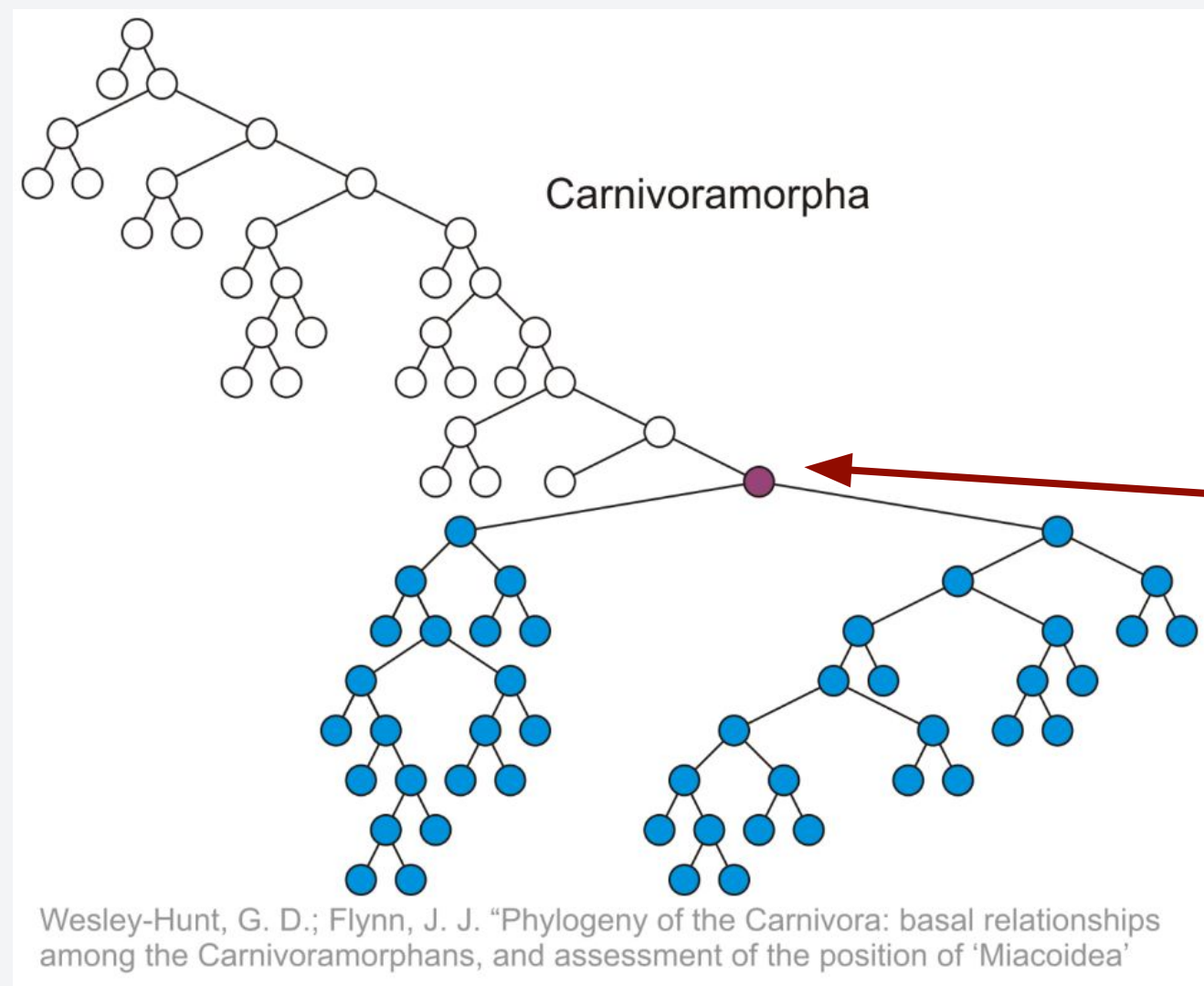


Wesley-Hunt, G. D.; Flynn, J. J. "Phylogeny of the Carnivora: basal relationships among the Carnivoramorpha, and assessment of the position of 'Miacoidea'"

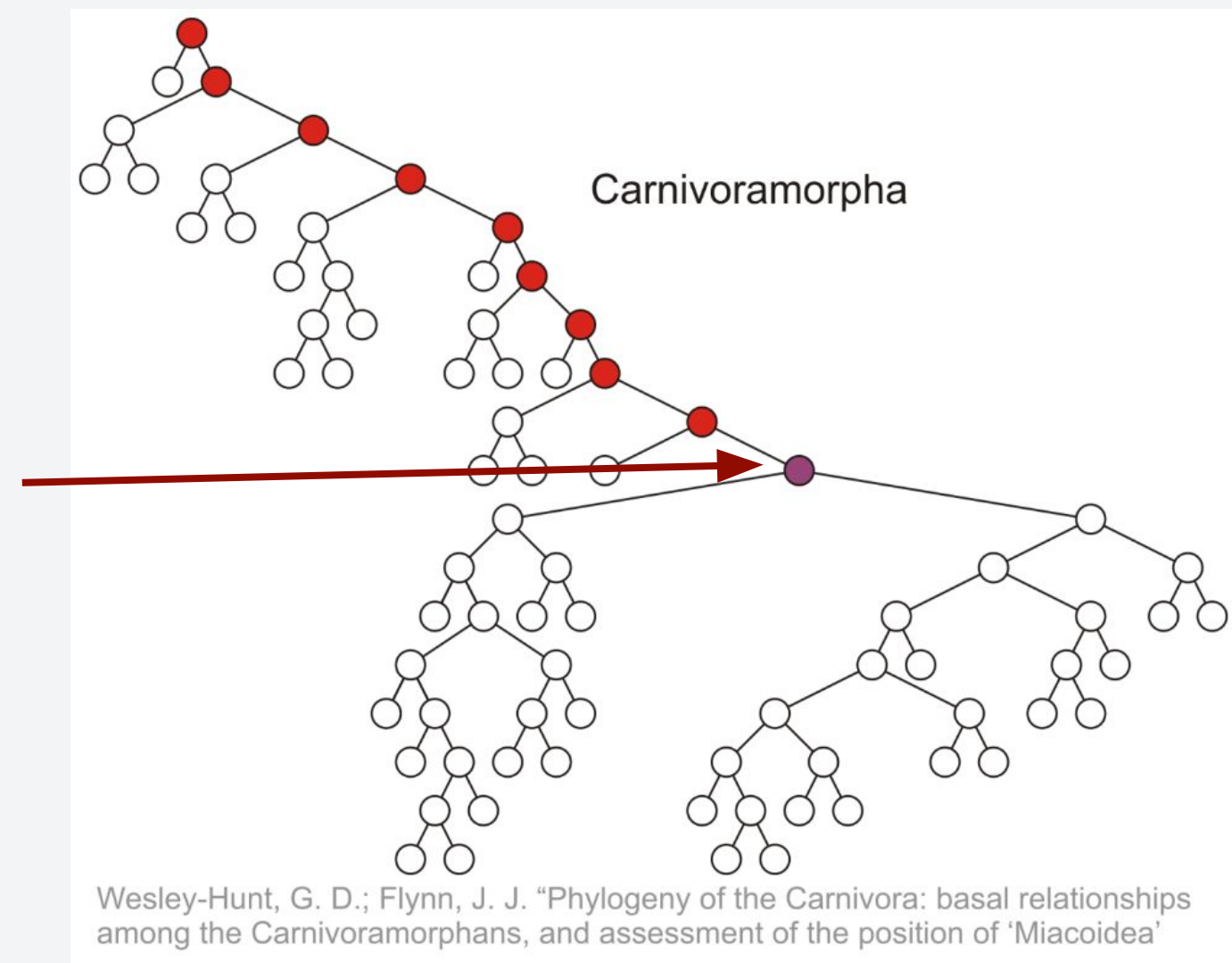
Tree Terminologies

With respect to a certain node,

- An **ancestor** is any element that is connected further up the tree.
- A **descendant** is any element that is connected lower down.



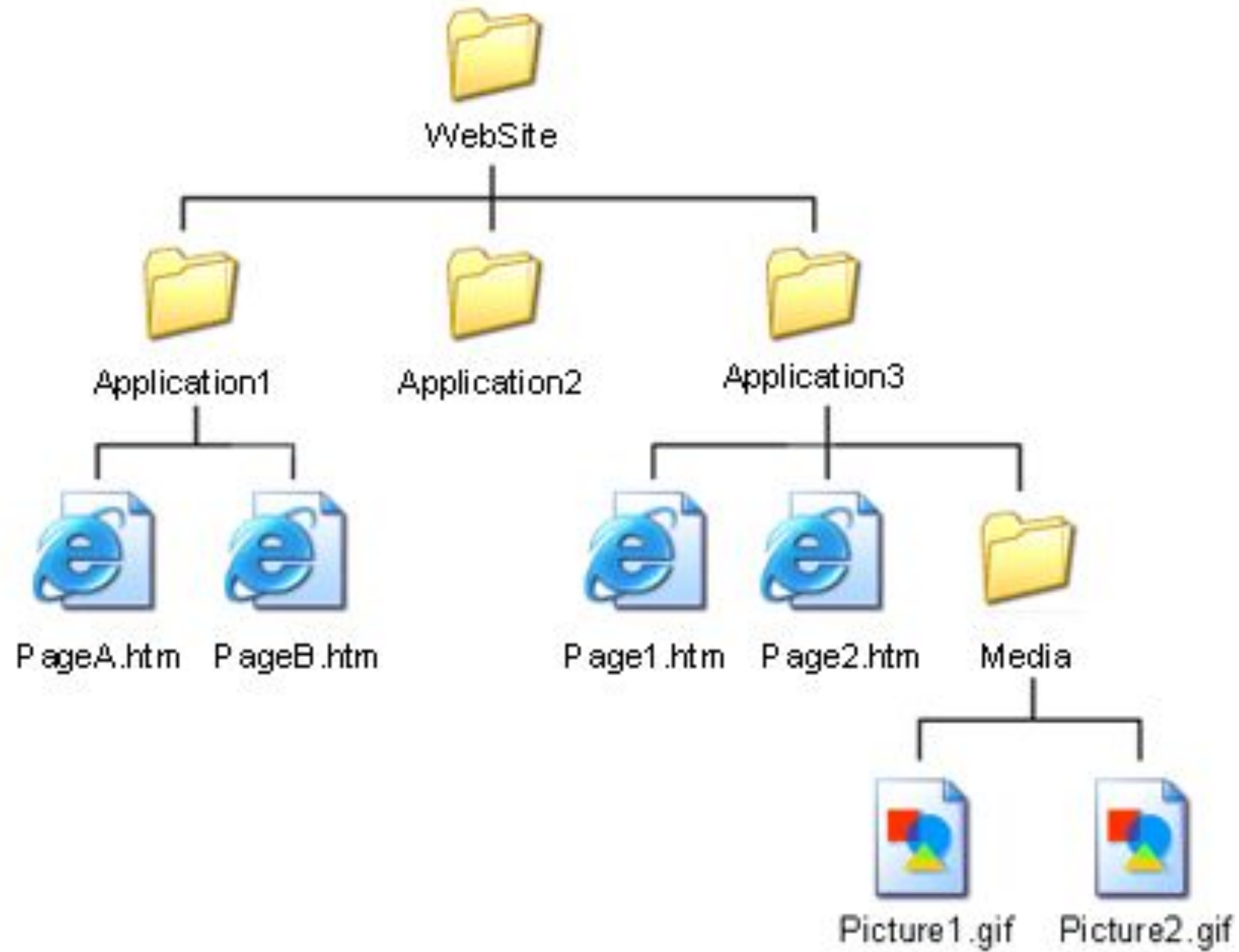
Ref.
Node



Trees in real life

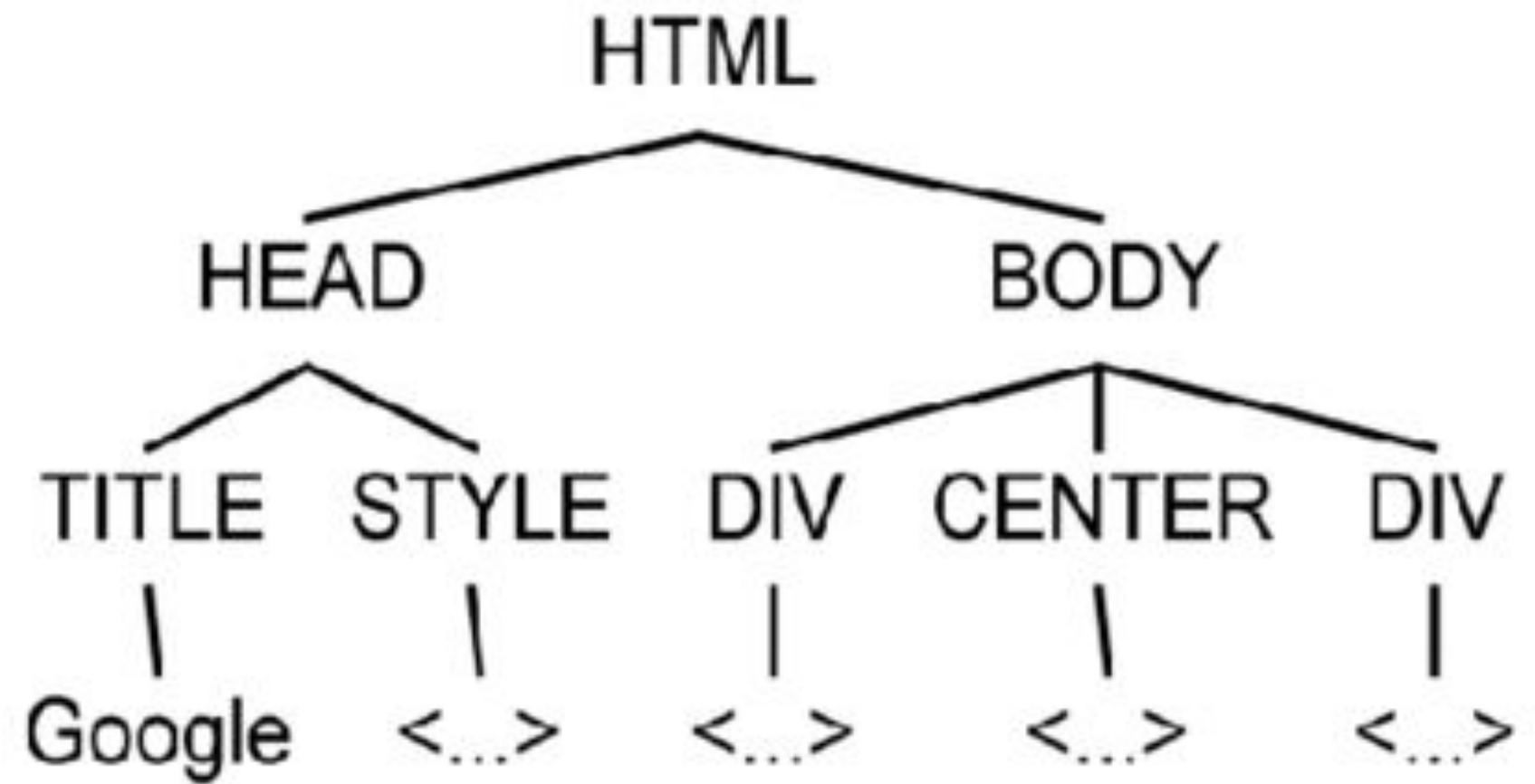


Trees in real life



Trees in real life

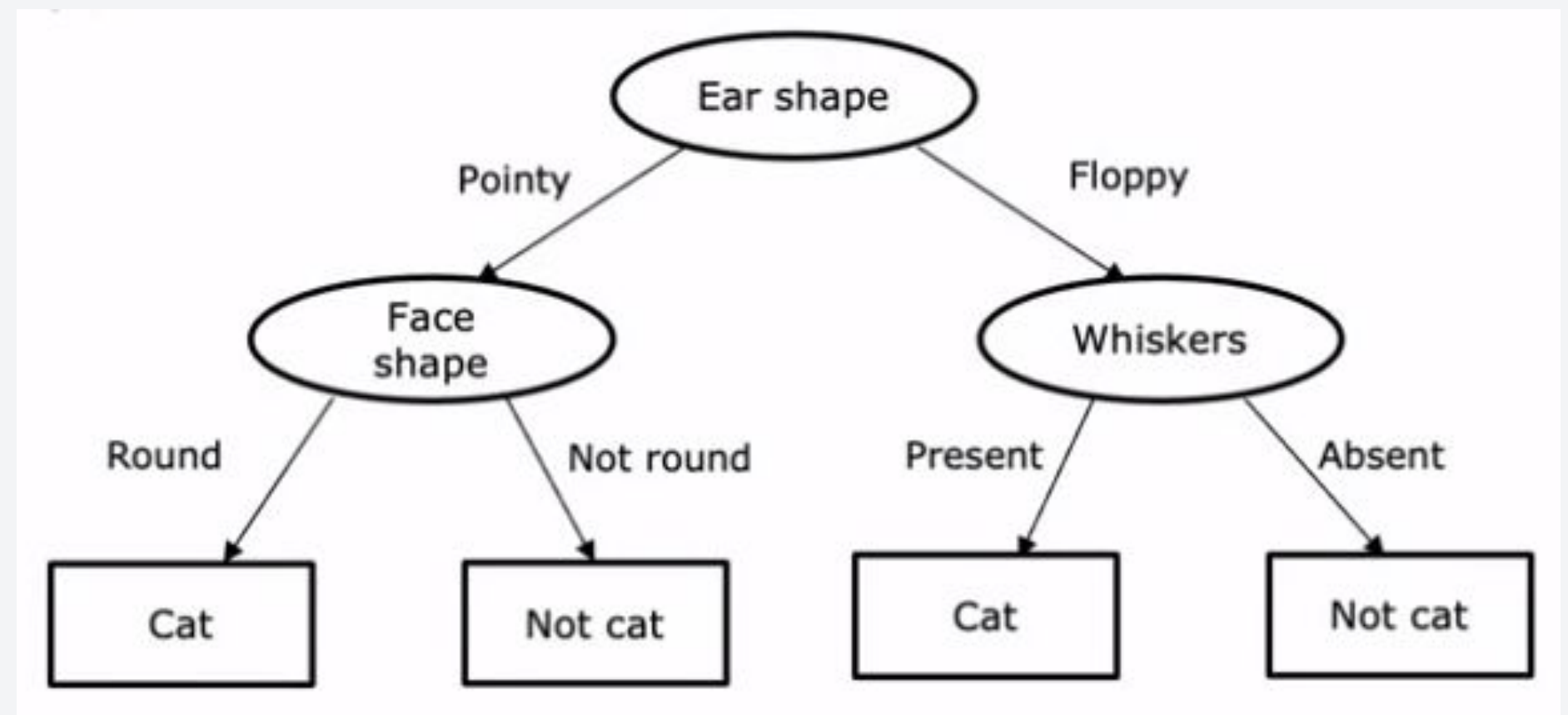
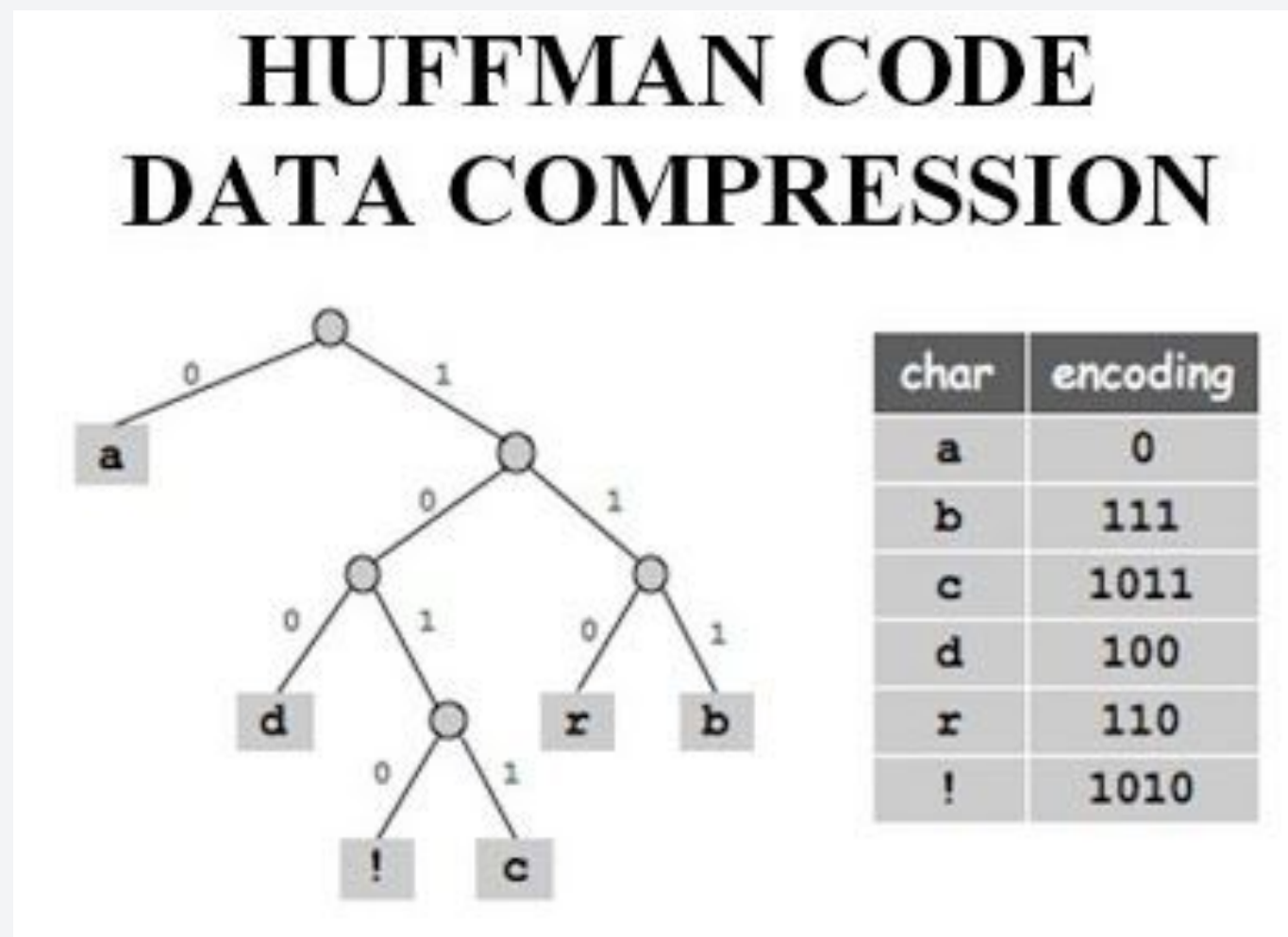
```
<html>
  <head>
    <title>Google</title>
    <style>...</style>
  </head>
  <body>
    <div>...</div>
    <center>...</center>
    <div>...</div>
  </body>
</html>
```



Binary Trees

In real life applications, using a tree with multiple children is often unnecessary. Most real-life applications are restricted to using nodes having only two branches.

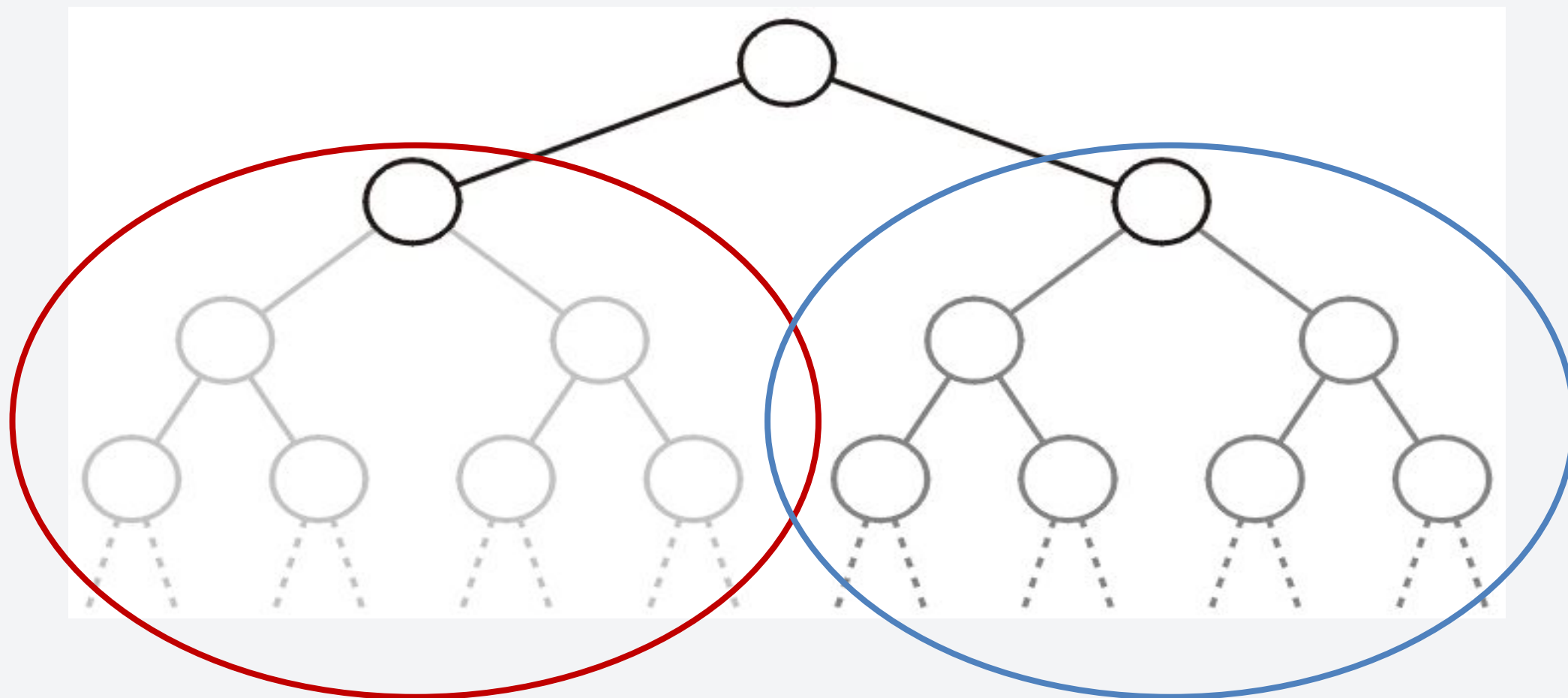
- Huffman Tree for Data Compression
- Decision Trees for Decision Making, Classification (Wow ML yarn?)



Binary Trees

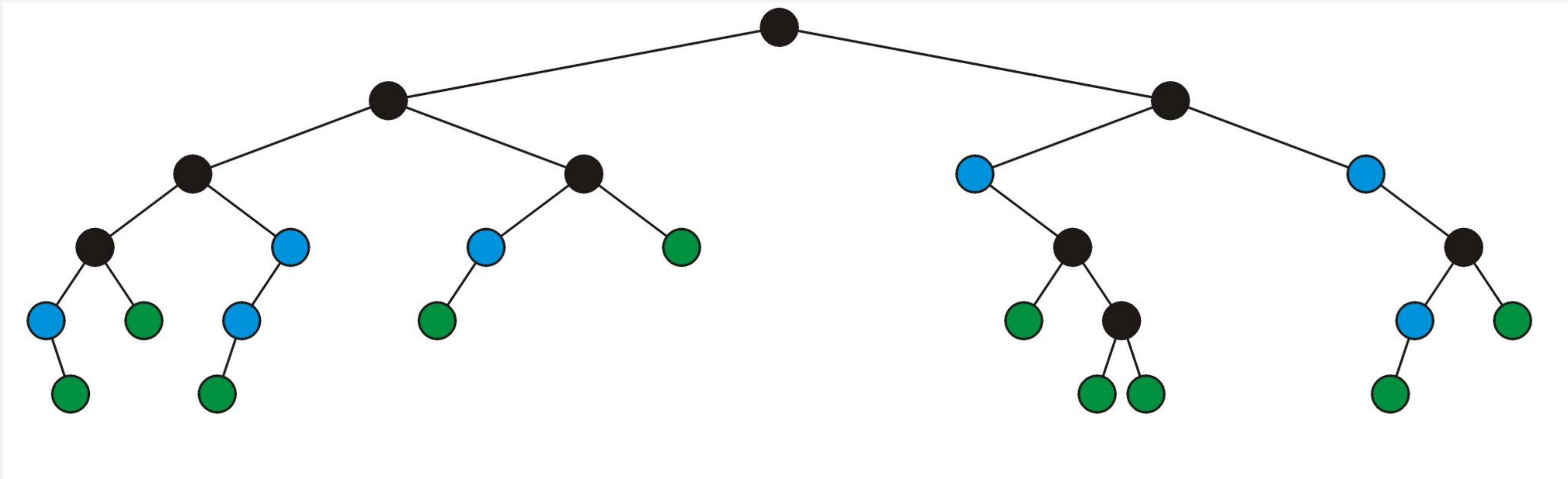
A binary tree is essentially a normal tree but with a restriction such that each node should only have **at most two children**.

- Each child is empty or another binary tree.
- We now have a **left** and **right** binary tree.



Binary Trees – Terminologies

- A full node is when both the left and right subtrees are non-empty.



Full Node



Neither



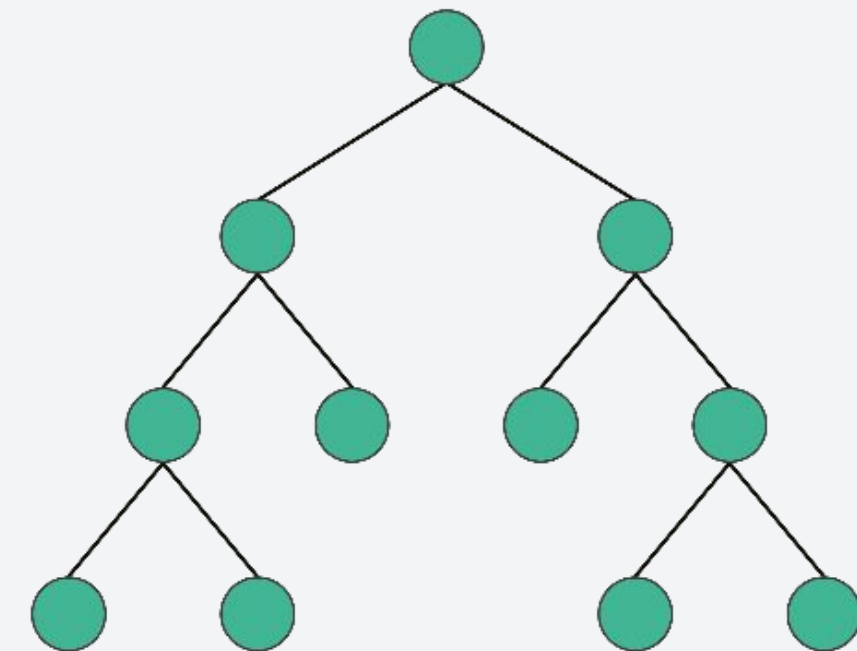
Leaf Nodes



Binary Trees – Terminologies

- **Full Tree**

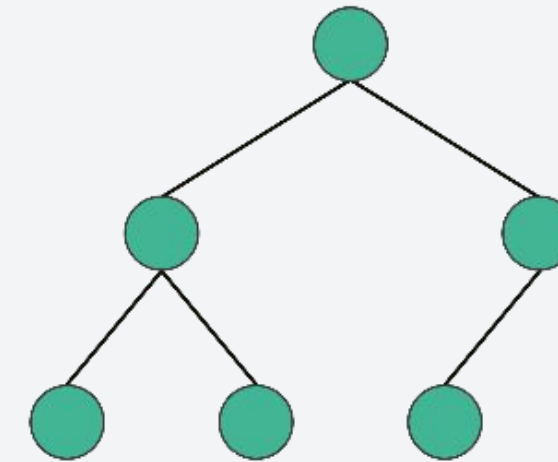
- All nodes except leaf nodes have two children.
- You either have 2 children or no children at all.



Full

Binary Trees – Terminologies

- **Complete Tree**
 - For every level, except the last level, is completely filled (left to right).

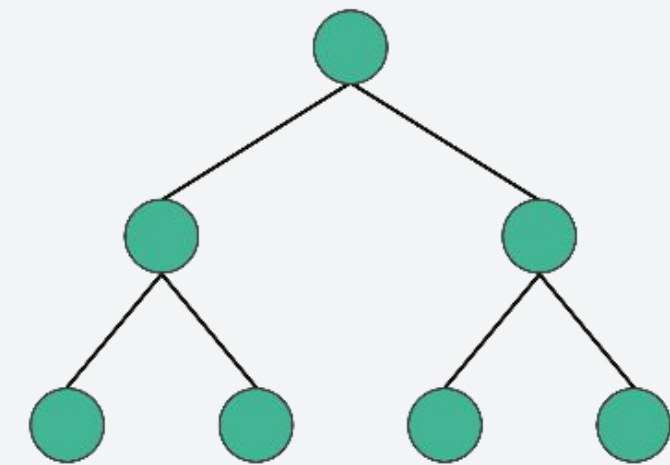


Complete

Binary Trees – Terminologies

- **Perfect Tree**

- Full and complete binary tree
- All nodes except the leaf nodes have two children and all leaf nodes are at the same level.

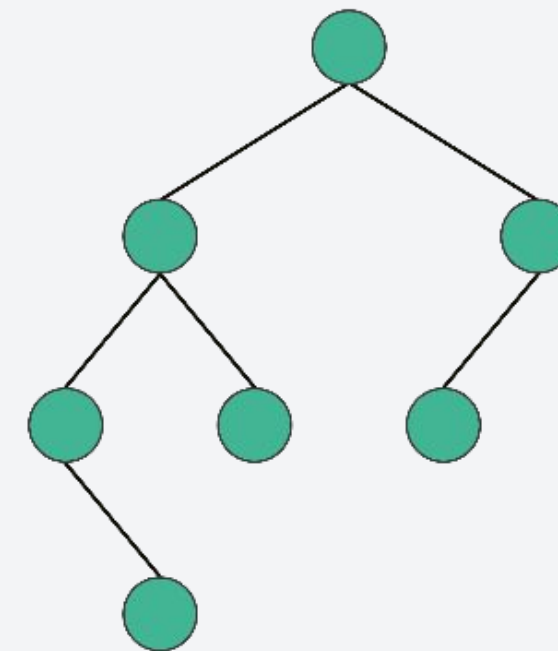


Perfect

Binary Trees – Terminologies

- **Balanced Tree**

- The difference between the height of the left and right subtree is less than or equal to 1.
- This condition has to be true for **all nodes** in the tree.

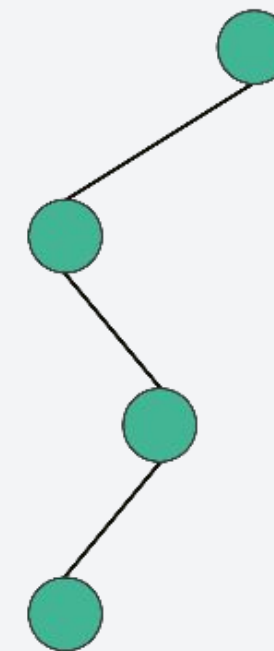


Balanced

Binary Trees – Terminologies

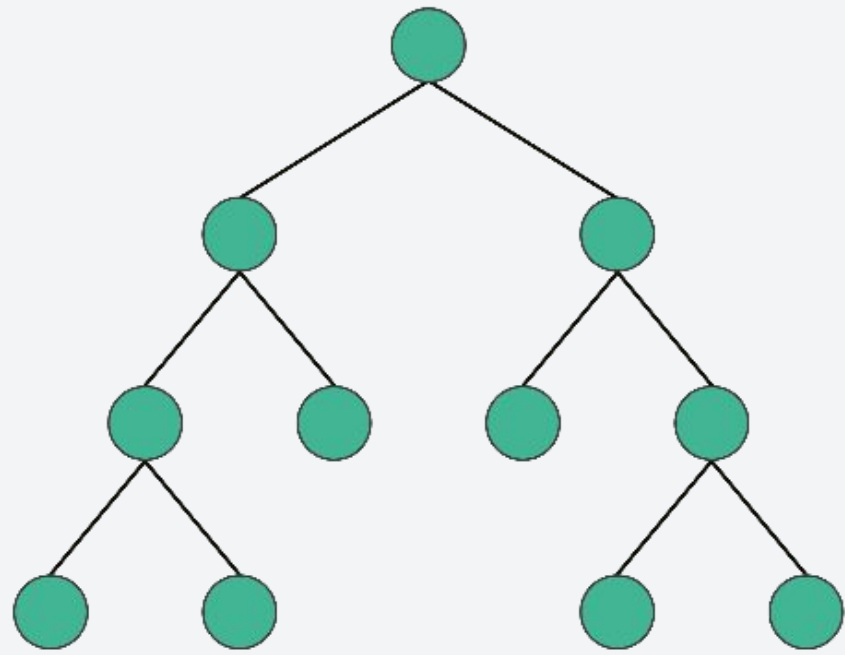
- **Degenerate**

- If each node only has one child resulting to a skewed tree resembling a linked list.
- Special Case: Skewed tree
 - A degenerate tree in which the tree is either dominated by the left nodes (left-skewed) or the right nodes (right-skewed).

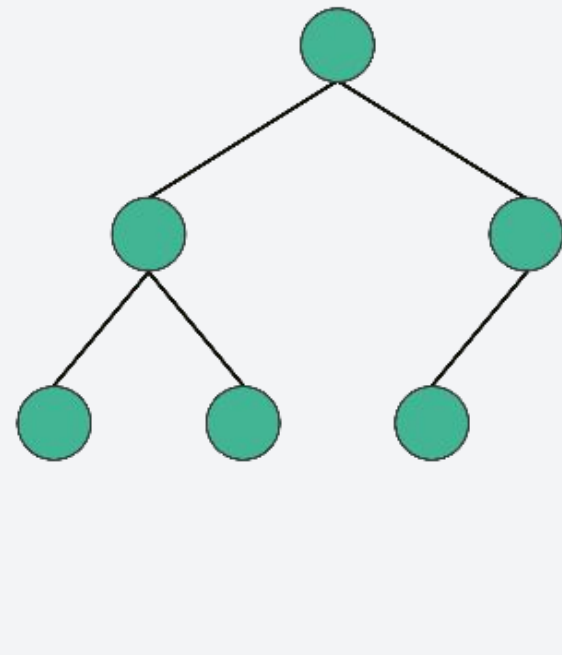


Degenerate

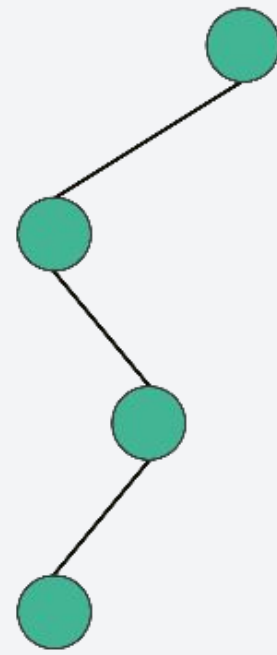
Binary Trees – Summary



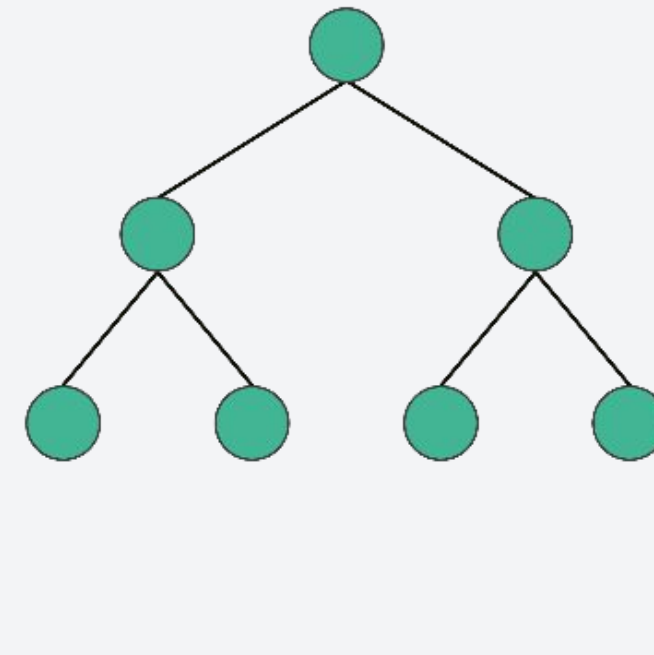
Full



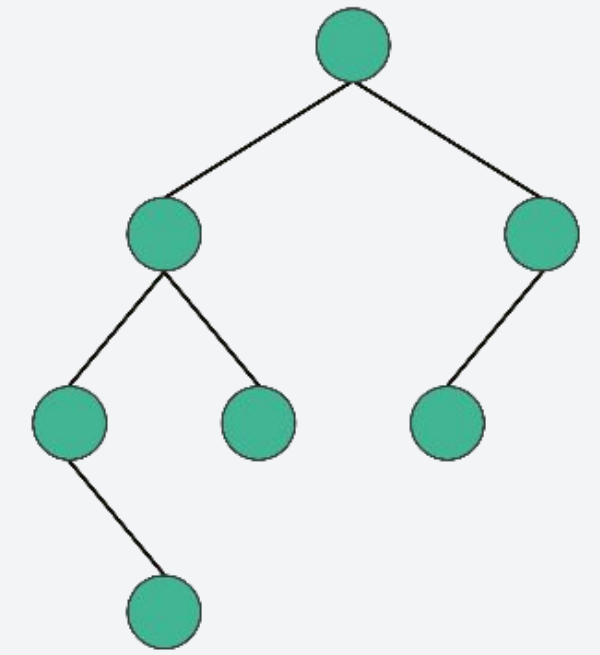
Complete



Degenerate



Perfect



Balanced

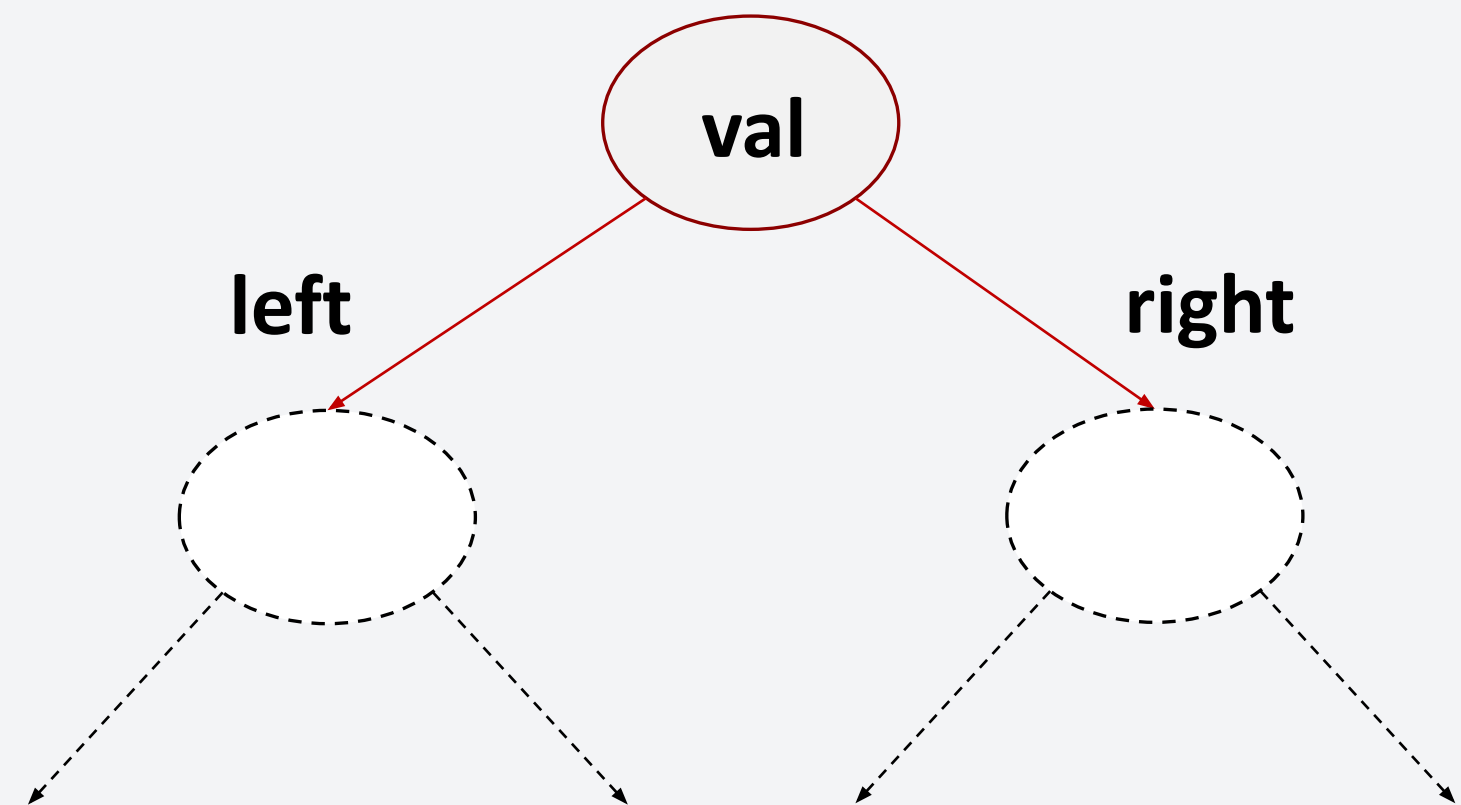
Binary Trees – Implementation

A binary needs to be able to:

- store a data
- link to the left and right subtrees (if it has children)

Similar to a linked list, we implement a node class

```
template<typename S>
class BinaryNode {
public:
    S val;
    BinaryNode* right;
    BinaryNode* left;
    BinaryNode(S val);
};
```



Binary Trees – Implementation

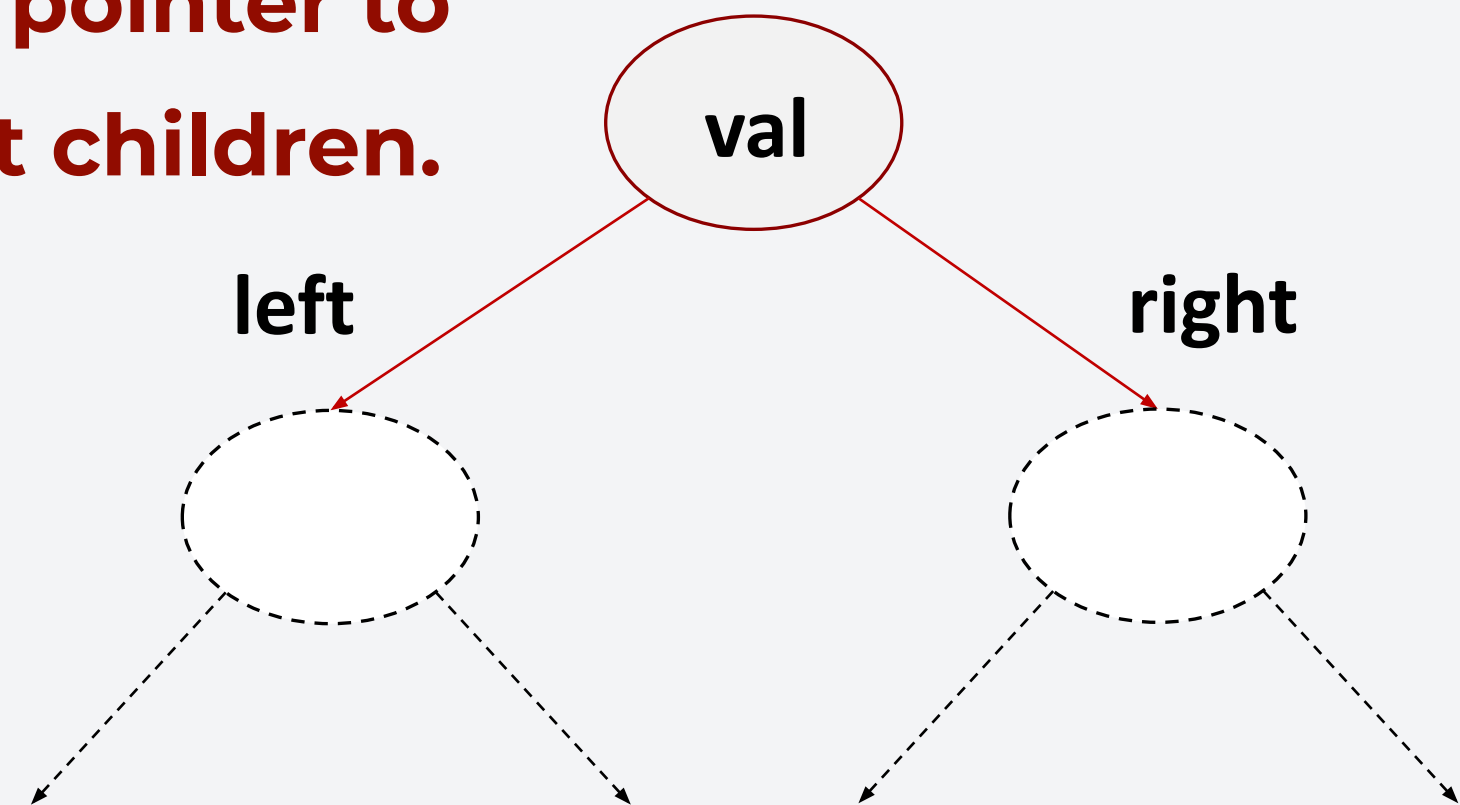
A binary needs to be able to:

- store a data
- link to the left and right subtrees (if it has children)

Similar to a linked list, we implement a node class

```
template<typename S>
class BinaryNode {
public:
    S val;
    BinaryNode* right;
    BinaryNode* left;
    BinaryNode(S val);
};
```

Each node has a pointer to the left and right children.



Binary Trees – Implementation

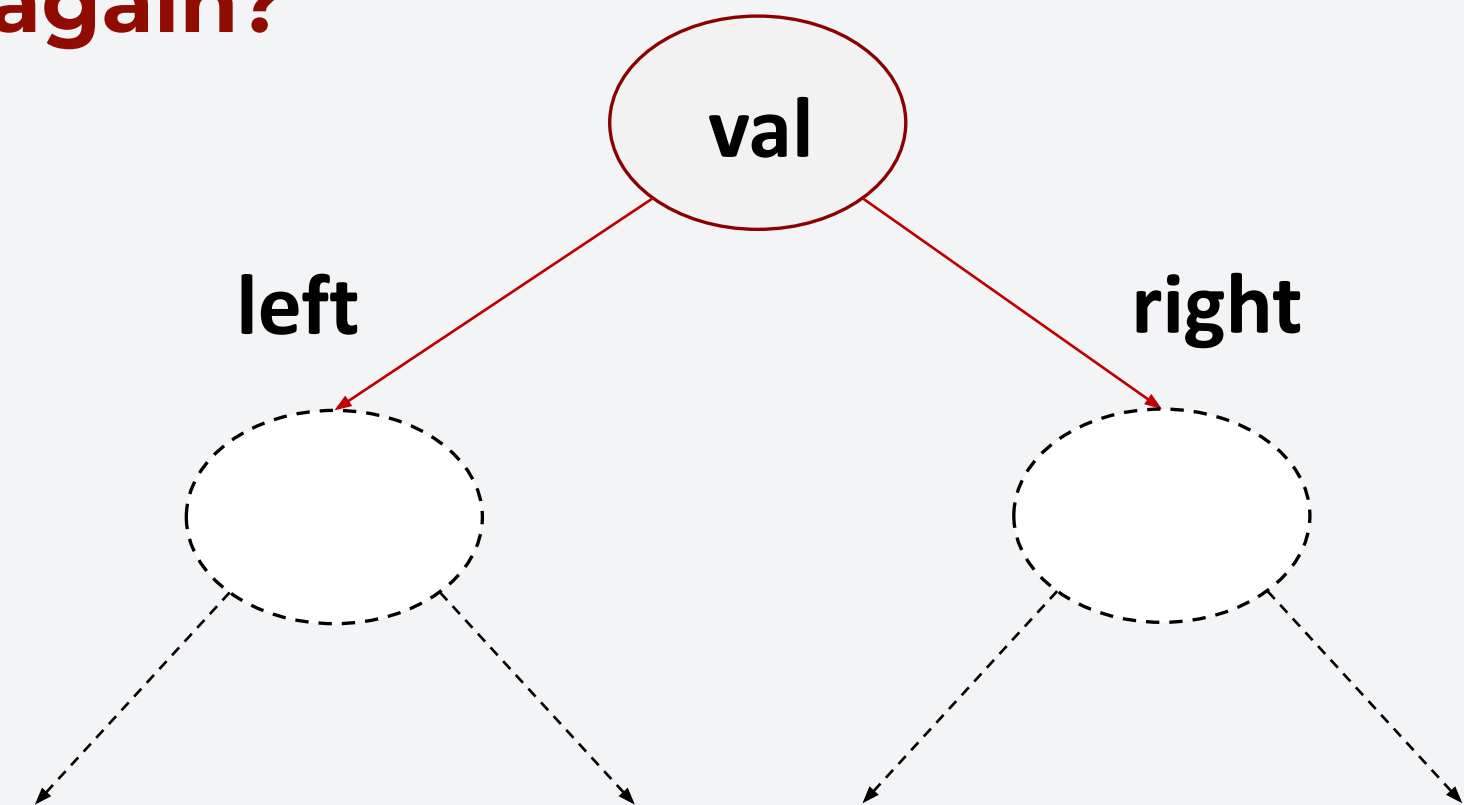
A binary needs to be able to:

- store a data
- link to the left and right subtrees (if it has children)

Similar to a linked list, we implement a node class

```
template<typename S>  
class BinaryNode {  
    public:  
        S val;  
        BinaryNode* right;  
        BinaryNode* left;  
        BinaryNode(S val);  
};
```

What is this again?



Binary Trees – Implementation

Sample Usage

```
template<typename S>
class BinaryNode {...};

template<typename S>
BinaryNode<S>::BinaryNode(S val):right(NULL),left(NULL),val(val) {}
...
BinaryNode<int>* node1 = new BinaryNode<int>(5);
node1 -> right = new BinaryNode<int>(4);
node1 -> left = new BinaryNode<int>(2);
```

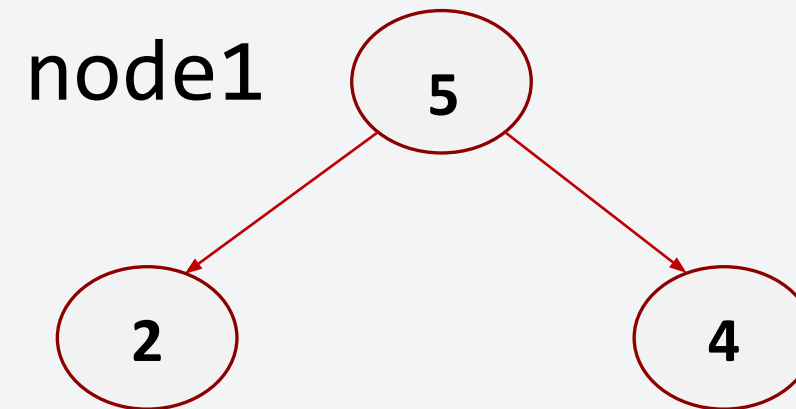

Binary Trees – Implementation

Sample Usage

```
template<typename S>  
class BinaryNode {...};
```

```
template<typename S>  
BinaryNode<S>::BinaryNode(S val):right(NULL),left(NULL),val(val) {}  
...
```

```
BinaryNode<int>* node1 = new BinaryNode<int>(5);  
node1 -> right = new BinaryNode<int>(4);  
node1 -> left = new BinaryNode<int>(2);
```



Binary Trees – Implementation

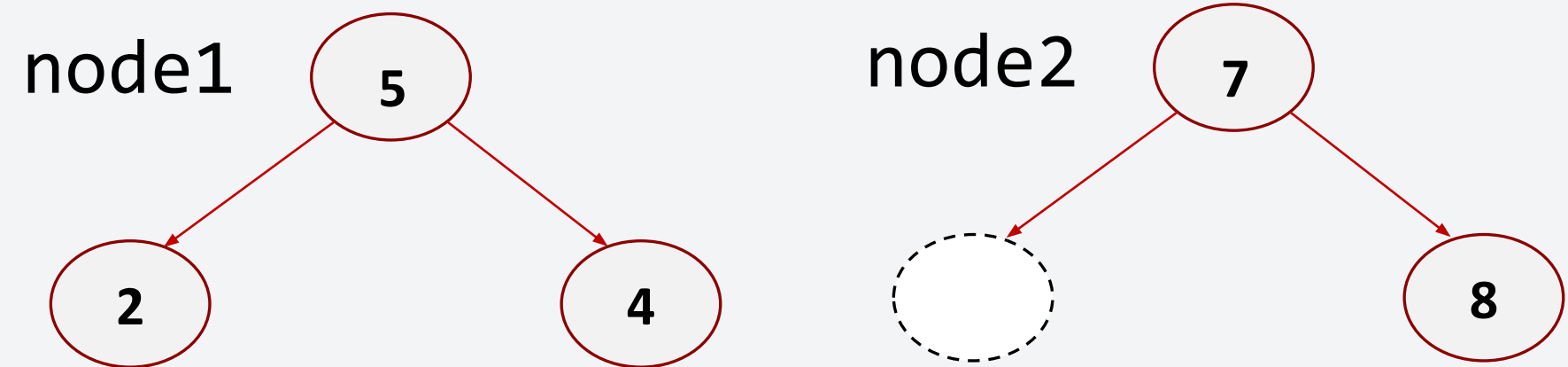
Sample Usage

```
template<typename S>  
class BinaryNode {...};
```

```
template<typename S>  
BinaryNode<S>::BinaryNode(S val):right(NULL),left(NULL),val(val) {}  
...
```

```
BinaryNode<int>* node1 = new BinaryNode<int>(5);  
node1 -> right = new BinaryNode<int>(4);  
node1 -> left = new BinaryNode<int>(2);
```

```
BinaryNode<int>* node2 = new BinaryNode<int>(7);  
node2 -> right = new BinaryNode<int>(8);
```



Binary Trees – Implementation

Sample Usage

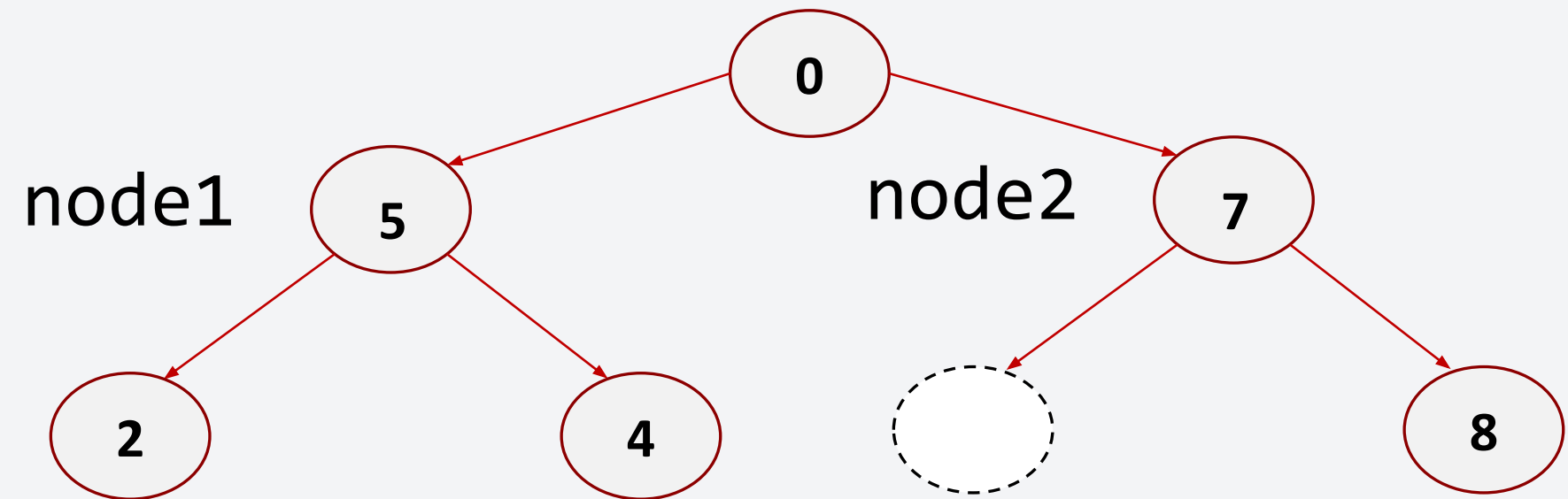
```
template<typename S>  
class BinaryNode {...};
```

```
template<typename S>  
BinaryNode<S>::BinaryNode(S val):right(NULL),left(NULL),val(val) {}  
...
```

```
BinaryNode<int>* node1 = new BinaryNode<int>(5);  
node1 -> right = new BinaryNode<int>(4);  
node1 -> left = new BinaryNode<int>(2);
```

```
BinaryNode<int>* node2 = new BinaryNode<int>(7);  
node2 -> right = new BinaryNode<int>(8);
```

```
BinaryNode<int>* root = new BinaryNode<int>(0);  
root -> left = node1;  
root -> right = node2;
```

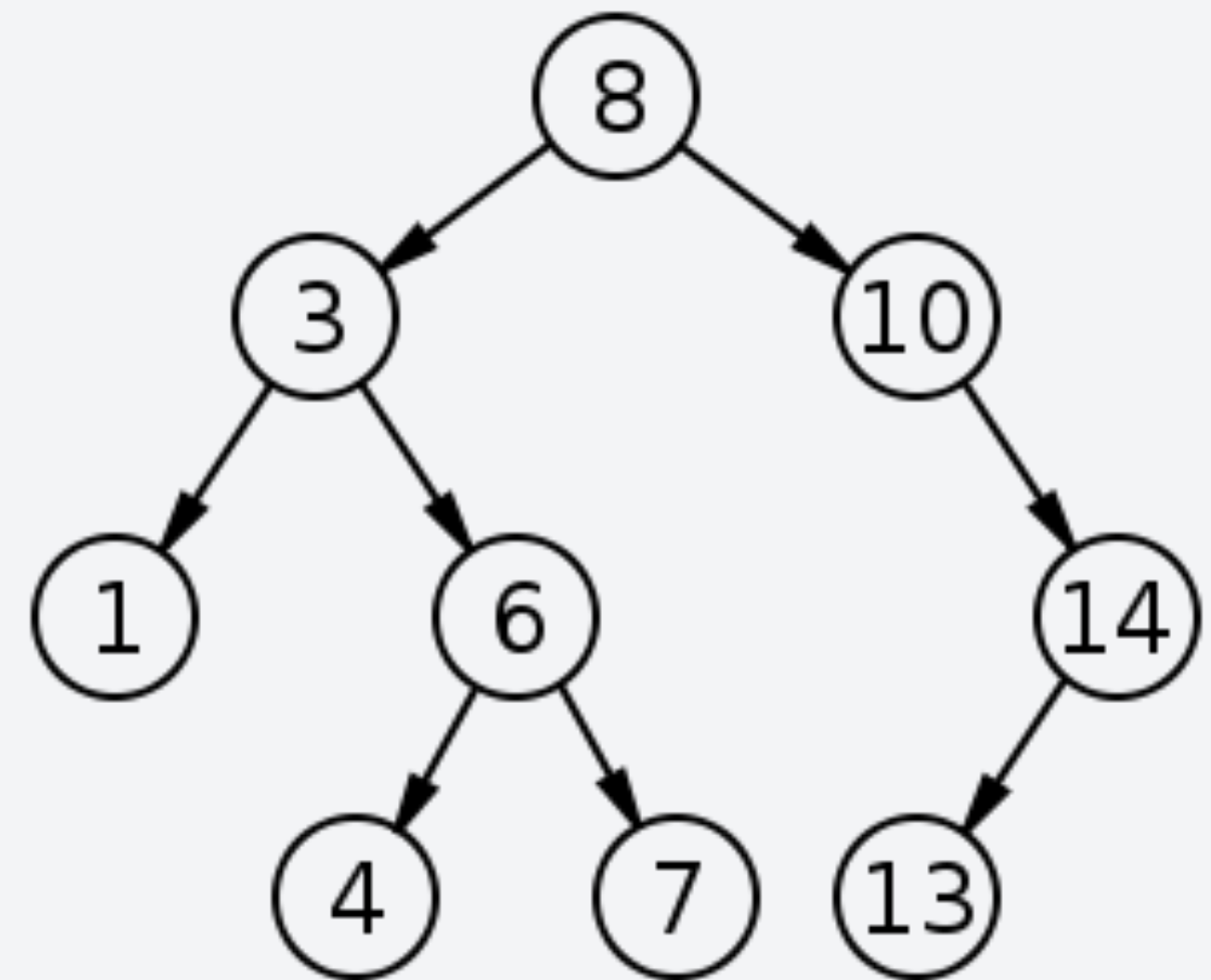


Binary Search Tree

A binary search tree is a type of binary tree that follows the properties below:

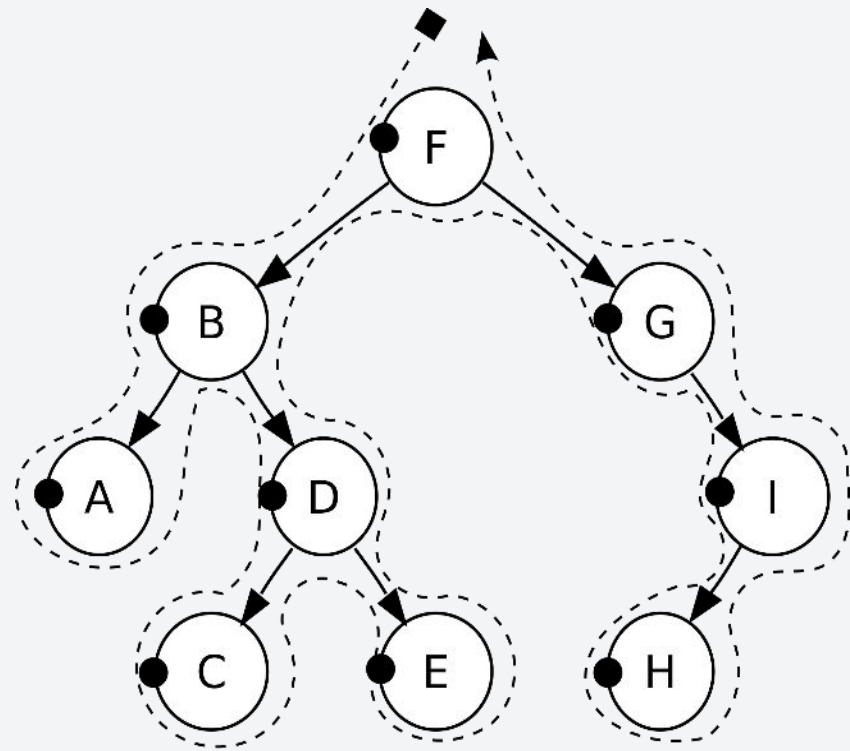
1. Each node contains a value.
2. The **value of the node on the left is always smaller** than the value of the current node.
3. The **value of the node on the right is always greater** than the value of the current node.

Suitable for needle-in-a-haystack problems (searching through a large search space given an ordered data).



Binary Search Tree – Tree Traversal

To perform a search in a binary search tree, we use a traversal algorithm.



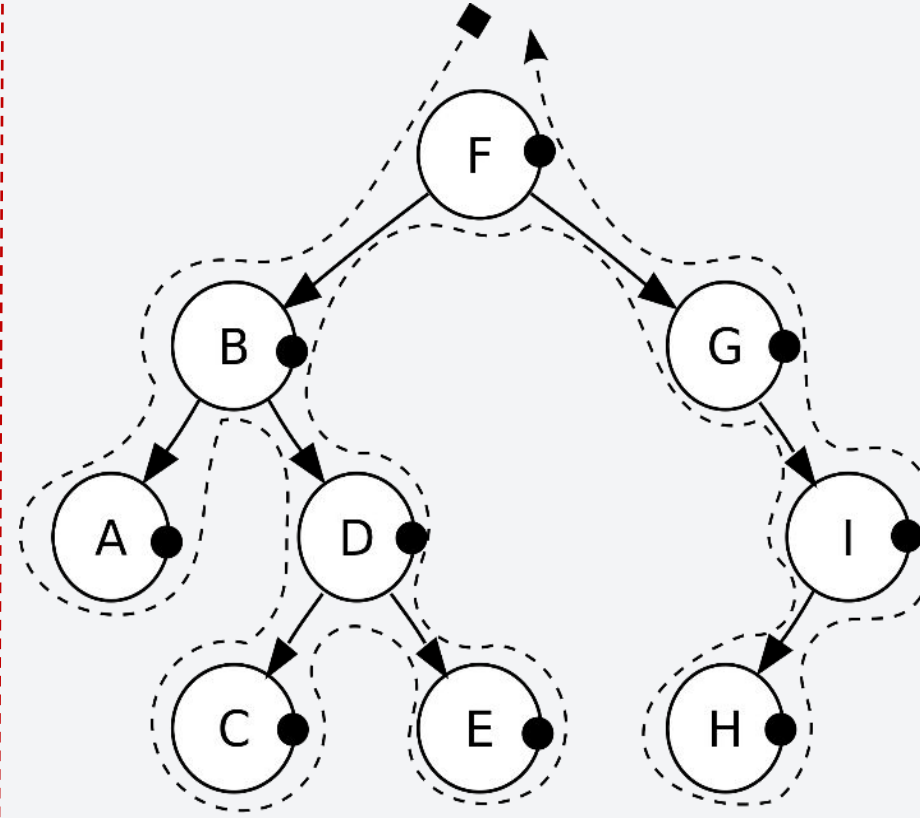
Pre-order: F B A D C E G I H

Preorder(node):

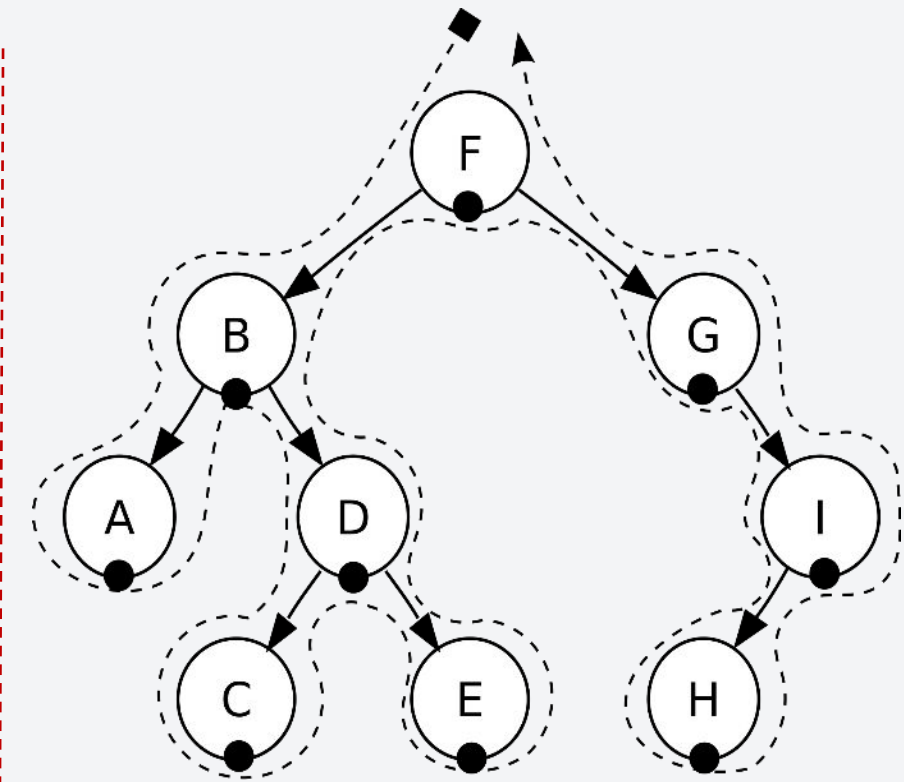
```
print(node -> value)
Preorder(node -> left)
Preorder(node -> right)
```

Postorder(node):

```
Postorder(node -> left)
Postorder(node -> right)
print(node -> value)
```



Pre-order: A C E D B H I G F



In-order: A B C D E F G H I

Inorder(node):

```
Inorder(node -> left)
print(node -> value)
Inorder(node -> right)
```


Binary Search Tree – Implementation

Class Implementation for BST

```
template<typename T>
class BST {
    private:
        BinaryNode<T>* root;
        int size;
        void addHelper(BinaryNode<T>* curr, T val);
        void deleteHelper(BinaryNode<T>* curr);
        void postorderHelper(BinaryNode<T>* curr);
        void preorderHelper(BinaryNode<T>* curr);
        void inorderHelper(BinaryNode<T>* curr);
    public:
        BST();
        void addNode(T val);
        void deleteTree();
        void postorder();
        void preorder();
        void inorder();
};
```



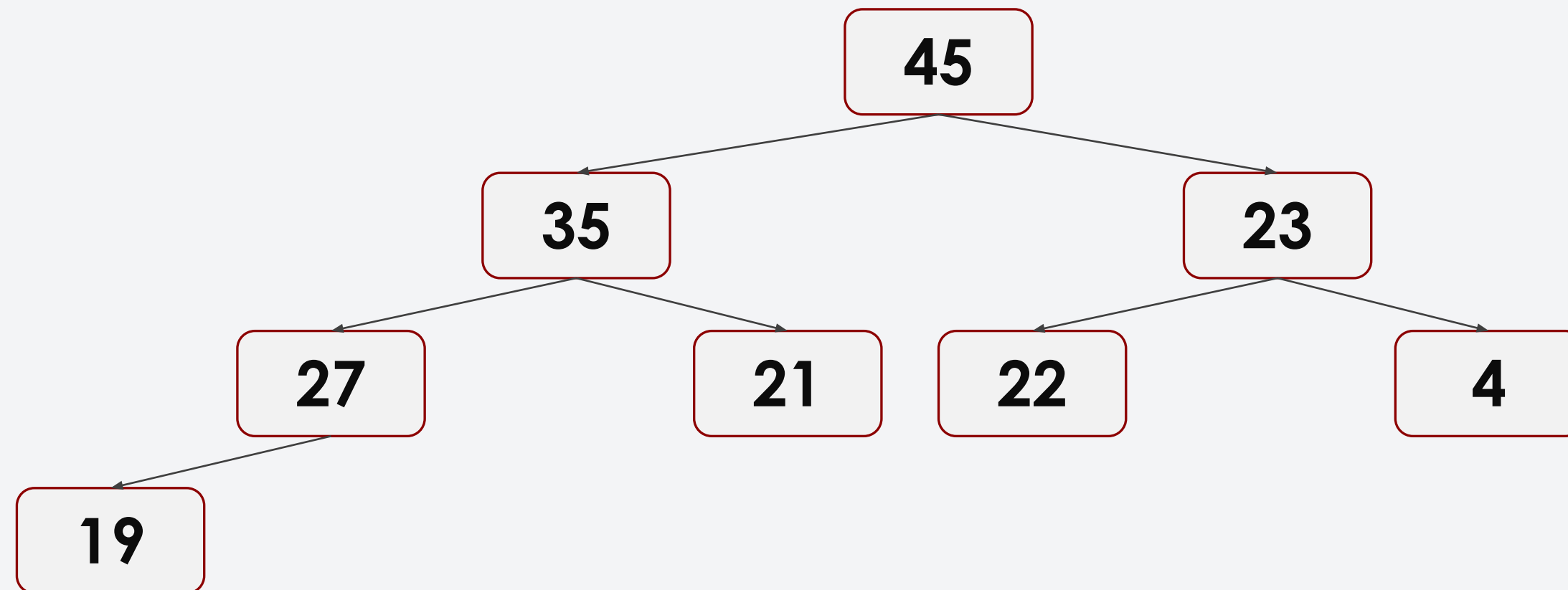
See Code 5-1 for
implementation details

Demo: Trees

Heap Data Structure

A heap is another variation of the binary tree that has two properties:

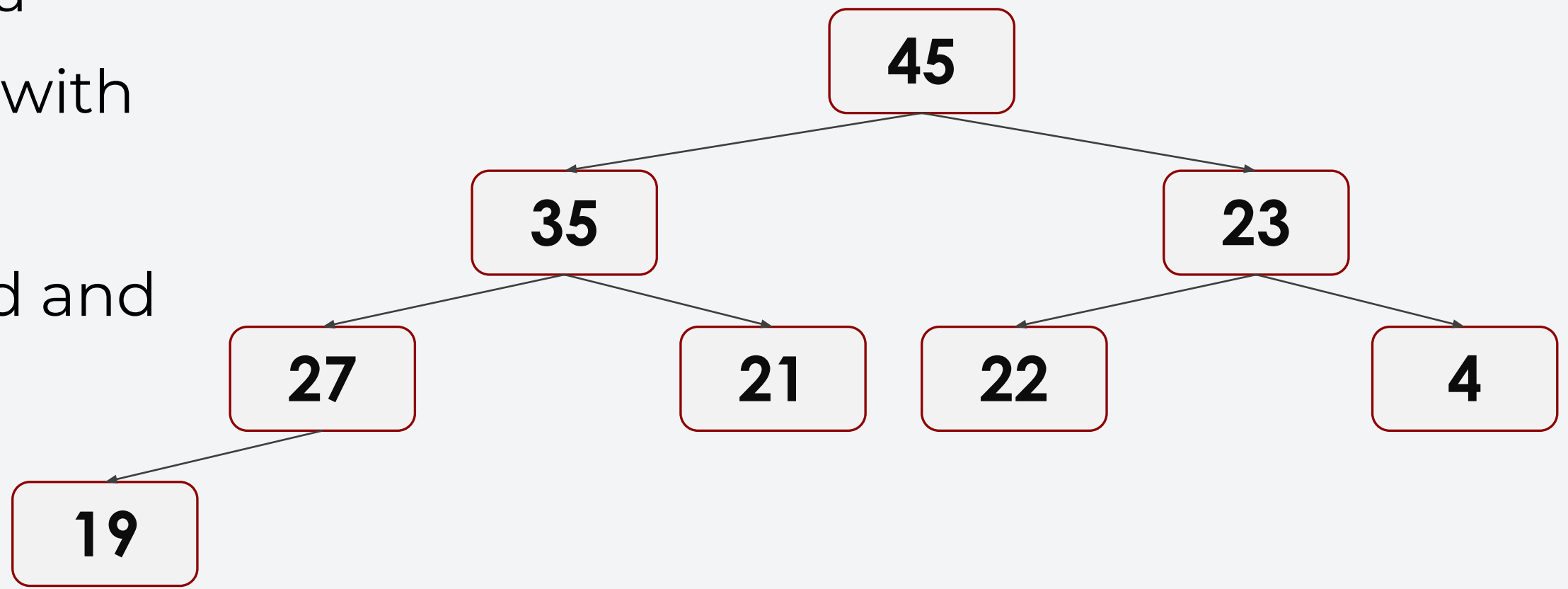
1. It is a complete binary tree.
2. A value or a key stored at a node is greater than or equal to the values of the child node.



Heap Data Structure

A heap is another variation of the binary tree that has two properties:

1. **Property 1:** It is a complete binary tree.
 2. **Property 2 (Heap Property):** The value stored at a node is greater than or equal to the values of the child node. (Max Heap; min heap is also possible :D)
- Each heap contains a key (or a value) that can be compared with other nodes.
 - We need a mechanism to add and remove elements.
 - Remember, the root has the largest element



Heap Data Structure – Methods

When creating the add and remove methods for the heap data structure, we need to remember its two properties:

1. A complete binary tree
2. The value of the parent node should always be larger than the child nodes

Adding an element:

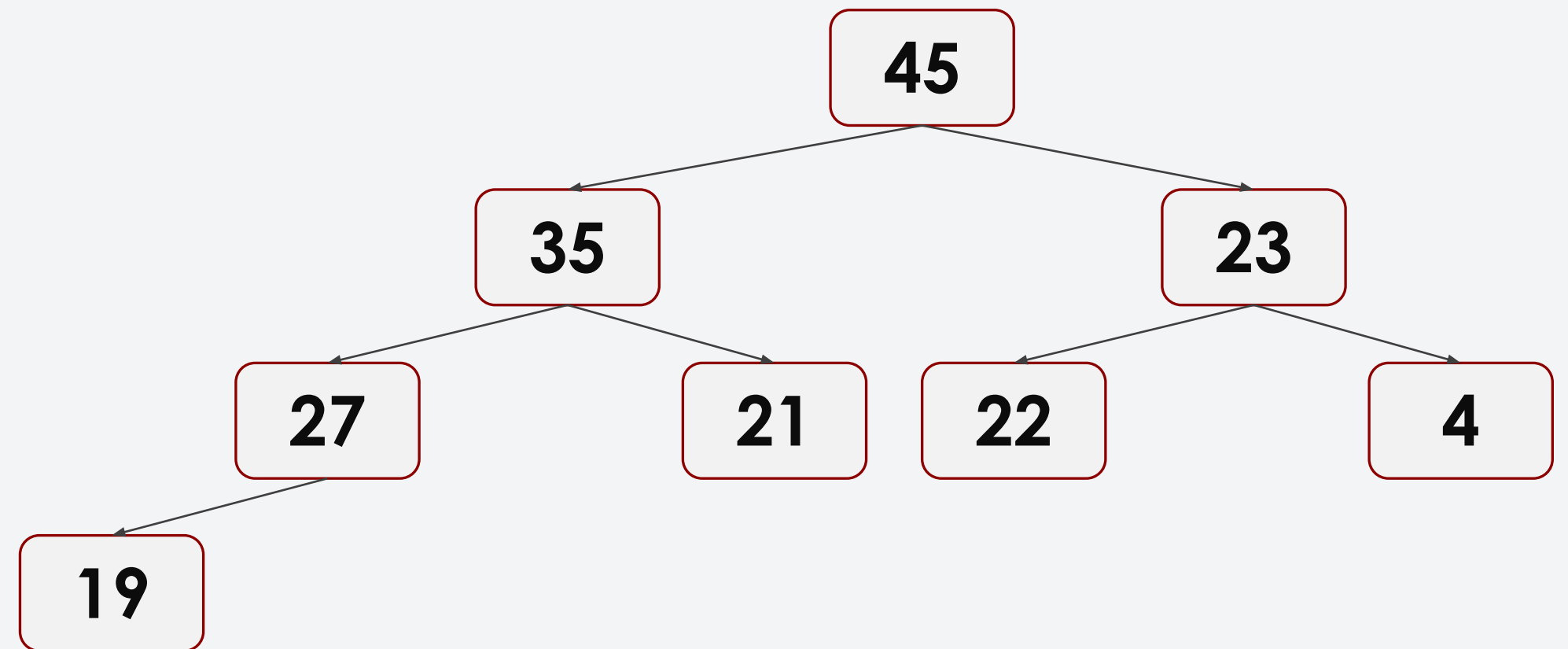
1. Put the new element in the next available spot.
2. Heapify the tree by rearranging the nodes to maintain the properties.

Removing the top element:

1. Remove the root and move the last node onto the root node's position.
2. Heapify the tree by rearranging the nodes to maintain the properties.

Heap Data Structure – Adding an element

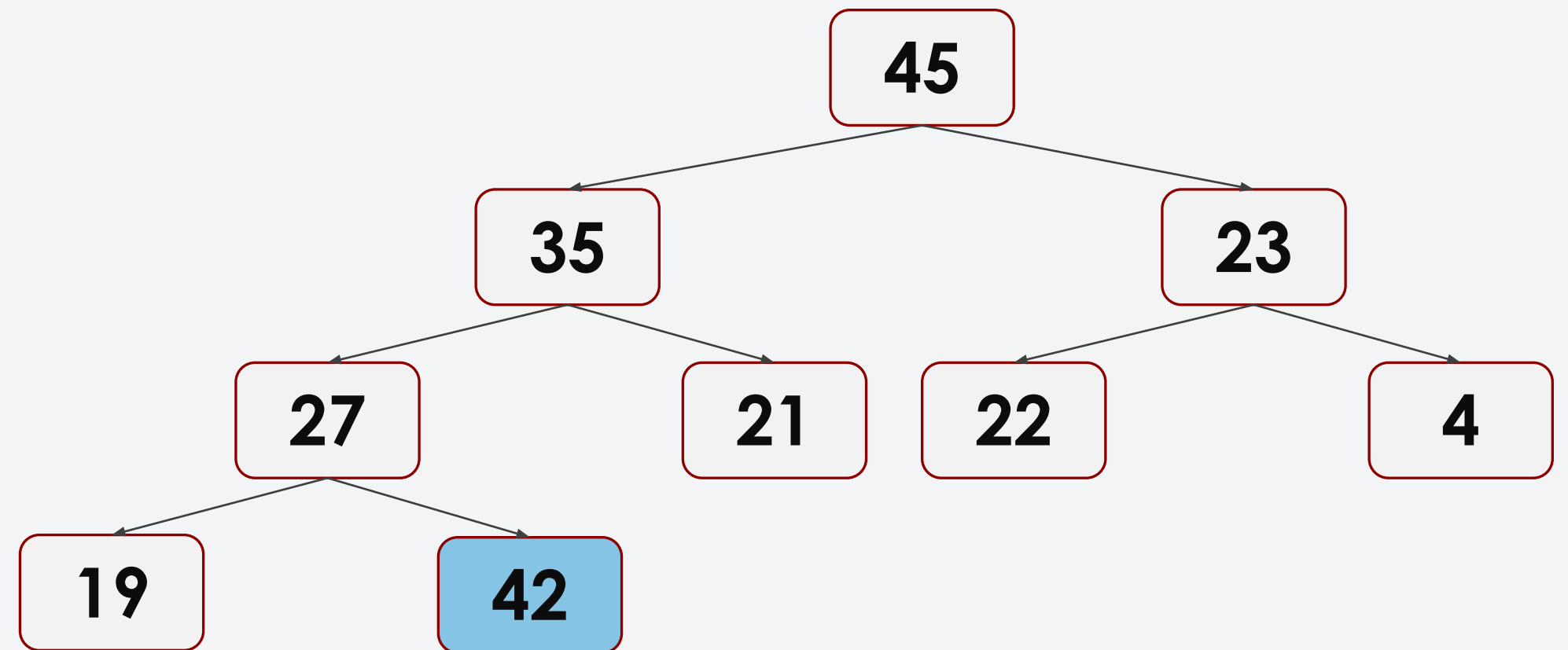
Suppose we want to add a new element with key equal to 42:



Heap Data Structure – Adding an element

Suppose we want to add a new element with key equal to 42:

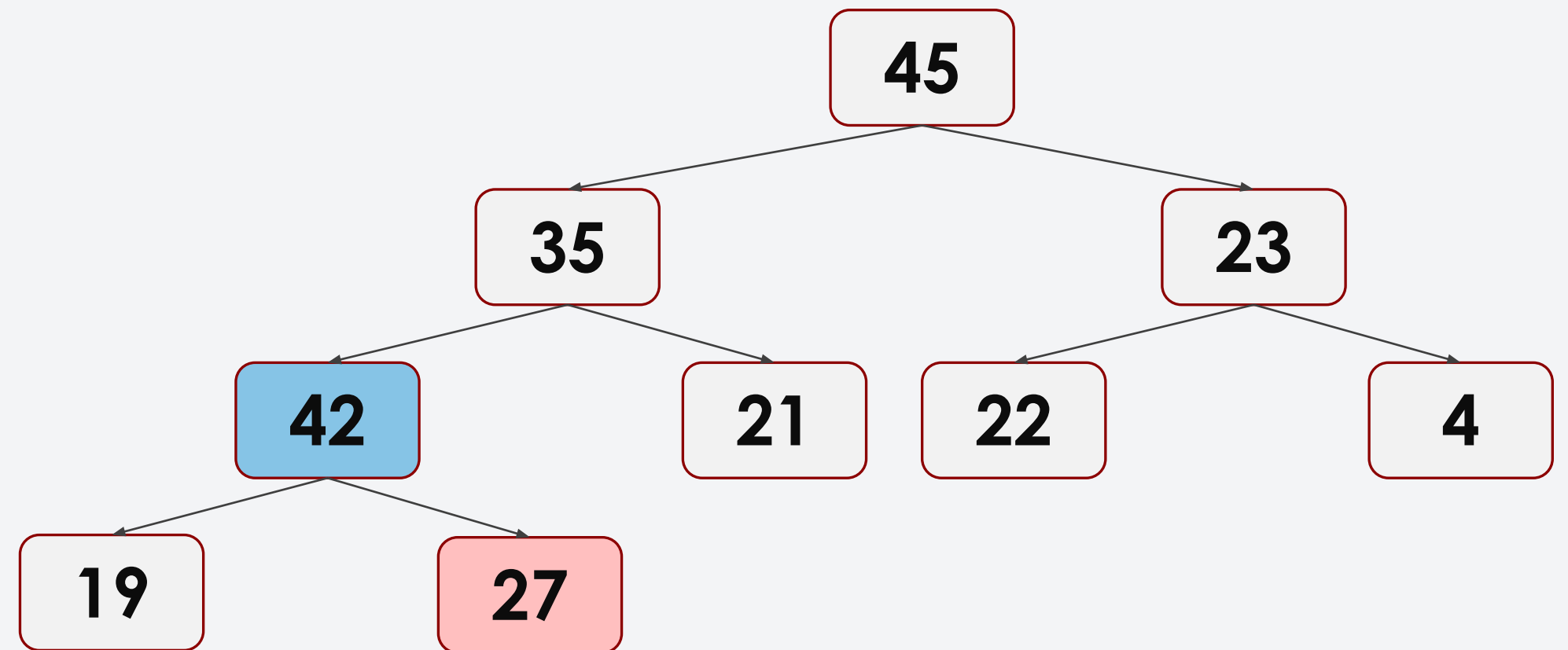
- Put the node in the next available spot.



Heap Data Structure – Adding an element

Suppose we want to add a new element with key equal to 42:

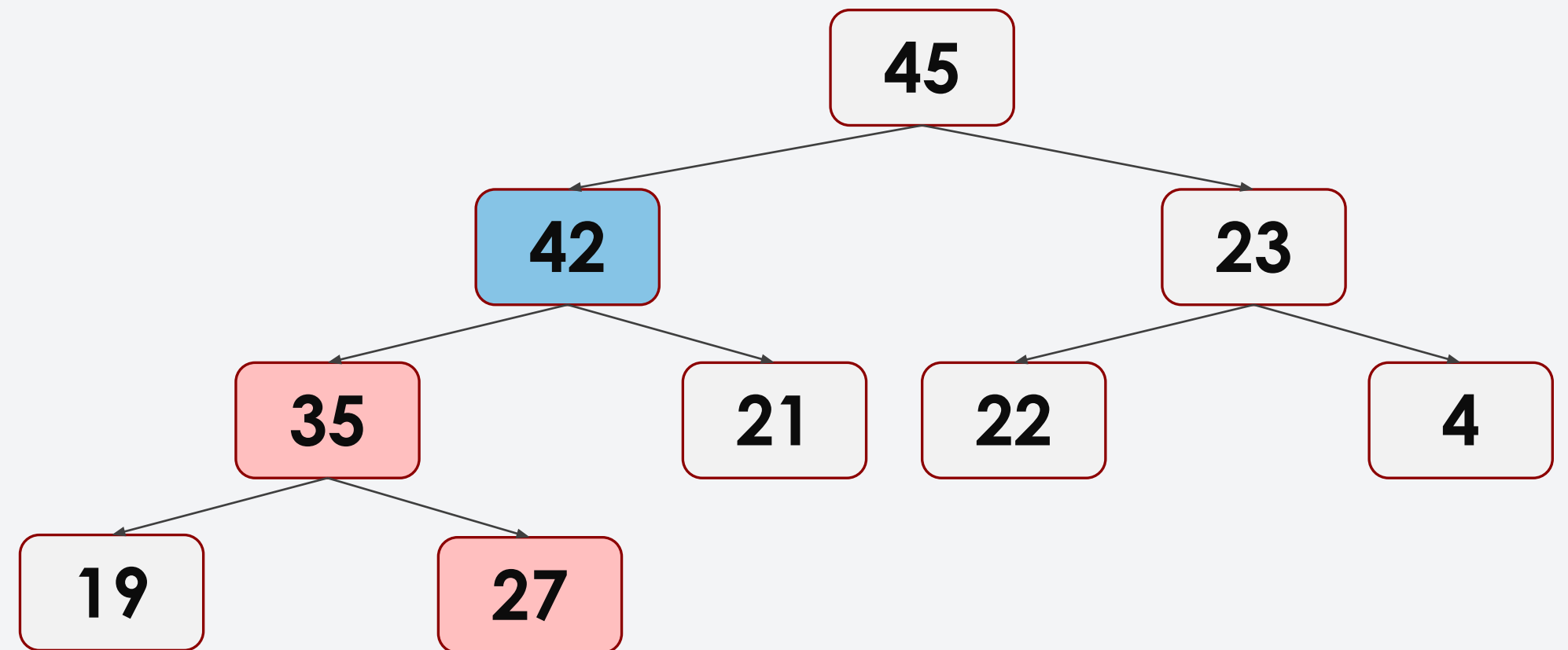
- Put the node in the next available spot.
- Push the new node upwards by swapping it with its parent.



Heap Data Structure – Adding an element

Suppose we want to add a new element with key equal to 42:

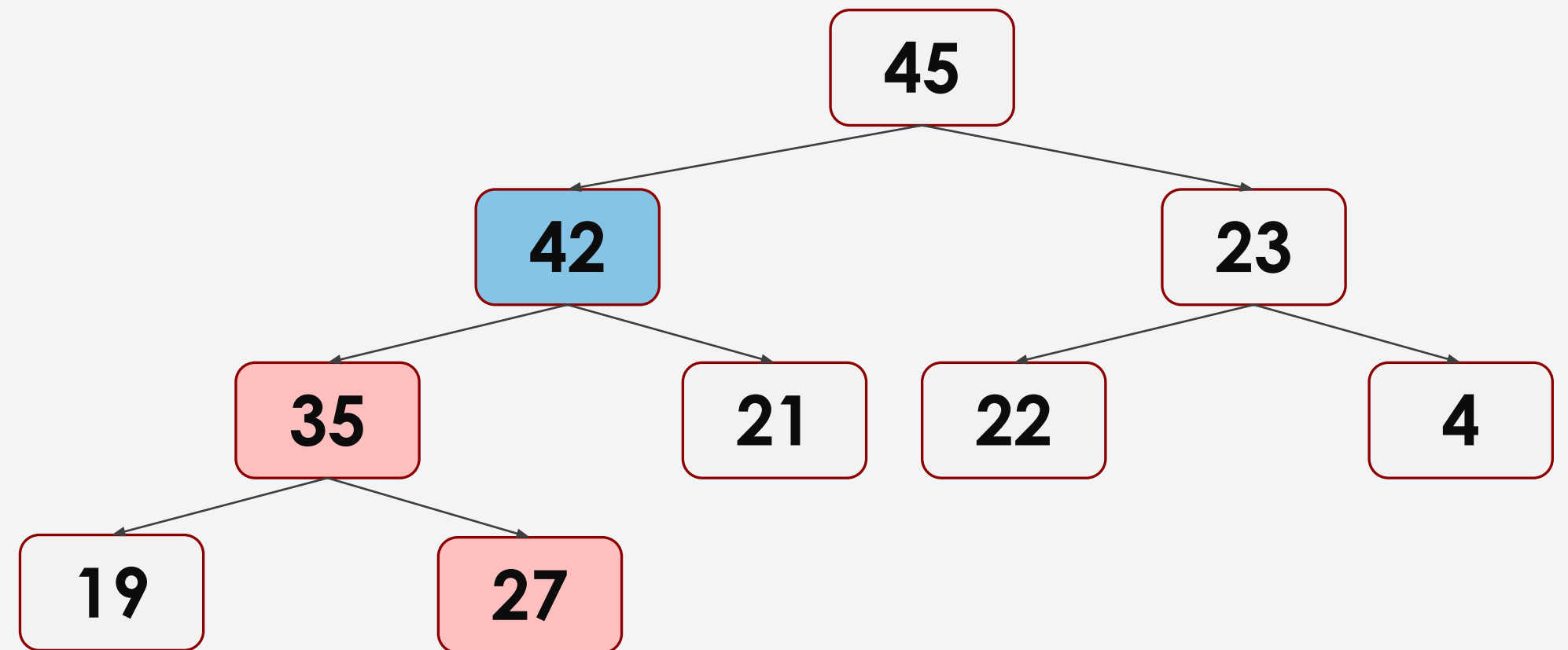
- Put the node in the next available spot.
- Push the new node upwards by swapping it with its parent.
- Repeat until:
 - The parent has a key $\geq$ the key of the new node (which is 42); or
 - The node reaches the root.



Heap Data Structure – Adding an element

Suppose we want to add a new element with key equal to 42:

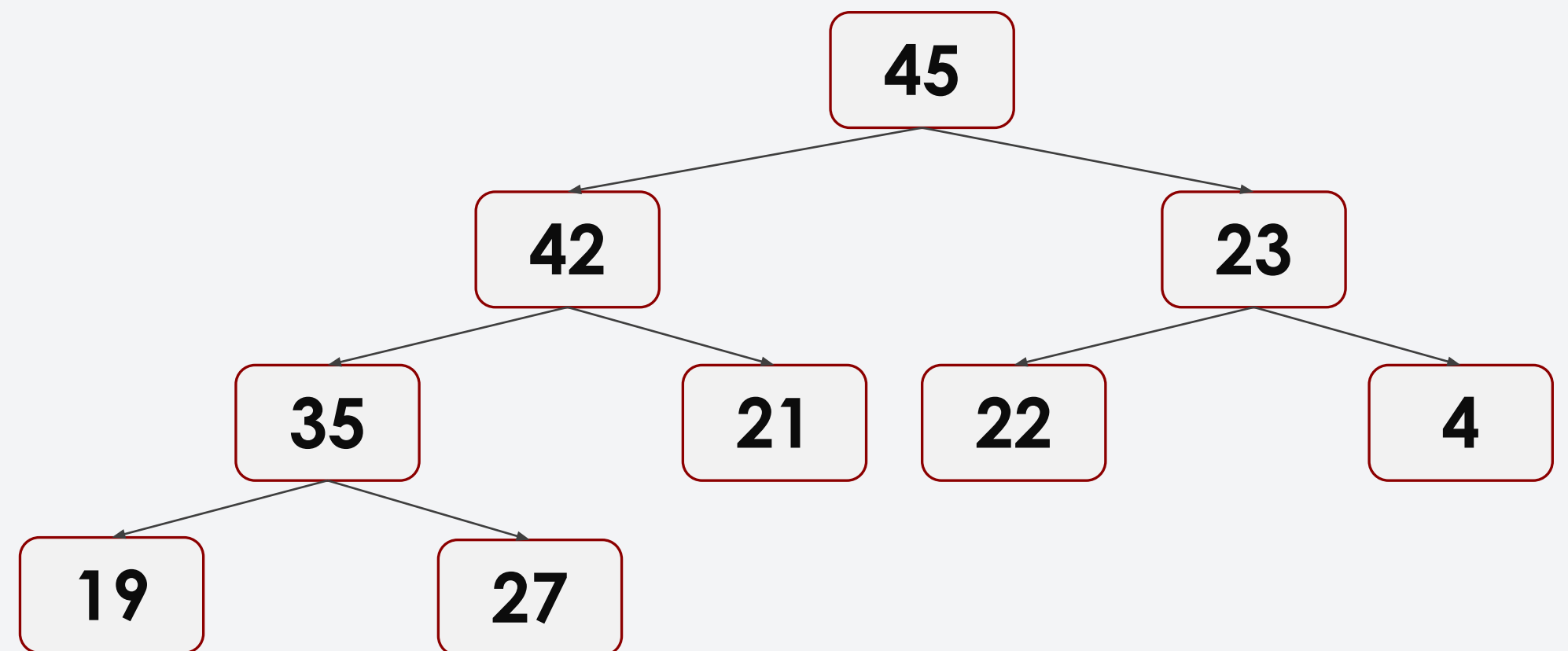
- Put the node in the next available spot.
- Push the new node upwards by swapping it with its parent.
- Repeat until:
 - The parent has a key $\geq$ the key of the new node (which is 42); or
 - The node reaches the root.



**This is called upwards
reheapification.**

Heap Data Structure – Removing an element

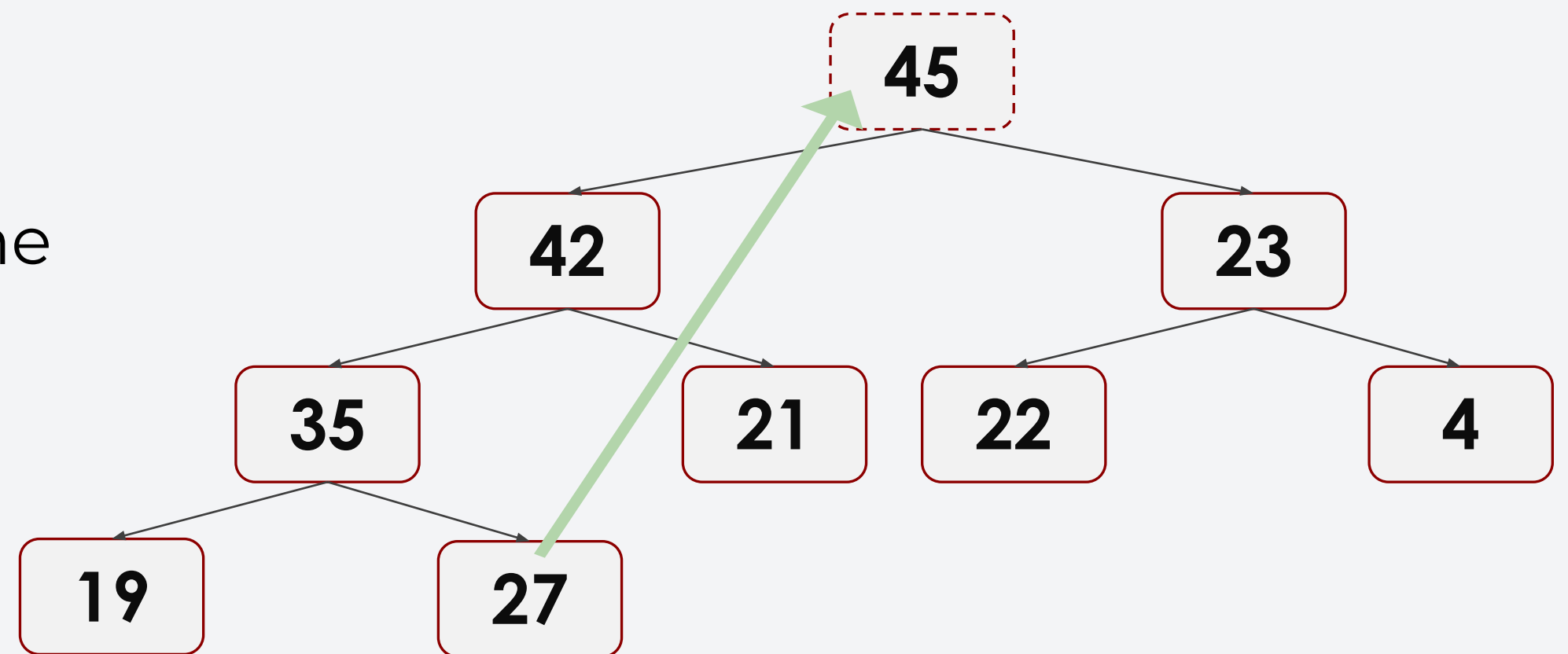
Suppose we want to remove the top element:



Heap Data Structure – Removing an element

Suppose we want to remove the top element:

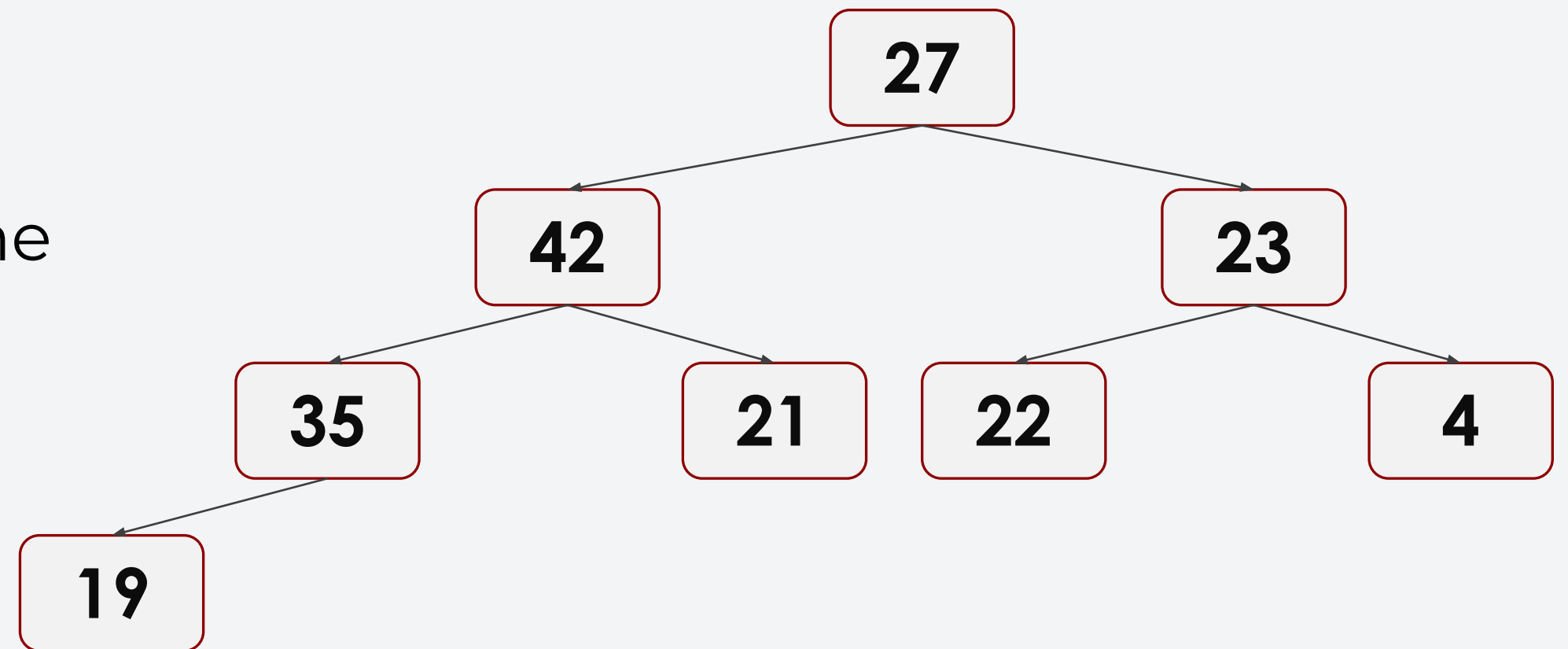
- Remove the root node and swap the last element of the heap.



Heap Data Structure – Removing an element

Suppose we want to remove the top element:

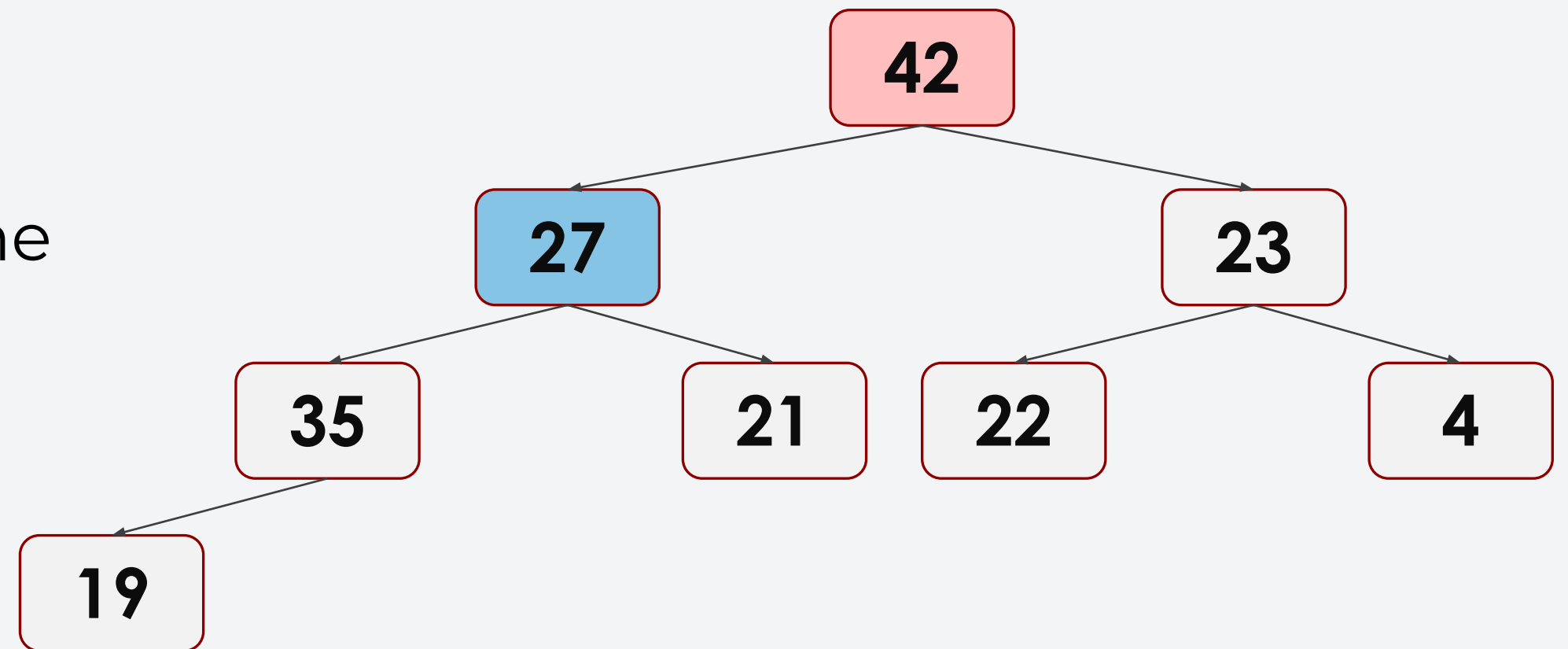
- Remove the root node and swap the last element of the heap.



Heap Data Structure – Removing an element

Suppose we want to remove the top element:

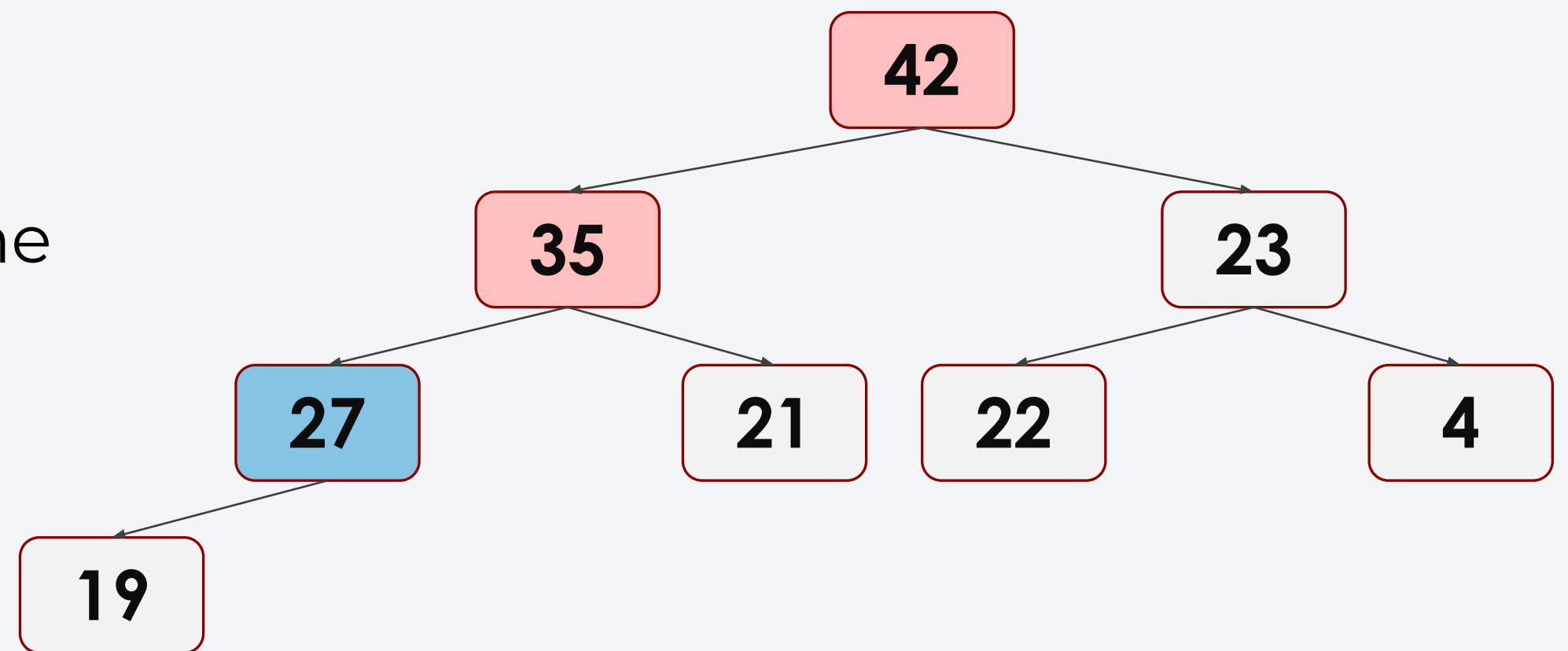
- Remove the root node and swap the last element of the heap.
- Push the out-of-place node downwards by swapping with its larger child.



Heap Data Structure – Removing an element

Suppose we want to remove the top element:

- Remove the root node and swap the last element of the heap.
- Push the out-of-place node downwards by swapping with its larger child.
- Repeat until the out-of-place node reaches an acceptable position.



**This is called downwards
reheapification.**

Heap Data Structure – Priority Queue

We saw that the heap data structure adds an element onto the heap and rearrange the heap to maintain its property. Additionally, we can only remove the top element which always has the largest key. Luckily, we have already have a C++ implementation for this called **priority queue**.

Initialization:

```
std::priority_queue<datatype> # Note: Max Heap by default
```

Methods:

- `push()`: Insert data into the heap
- `pop()`: remove data from the heap
- `top()`: return the top element (largest) of the heap

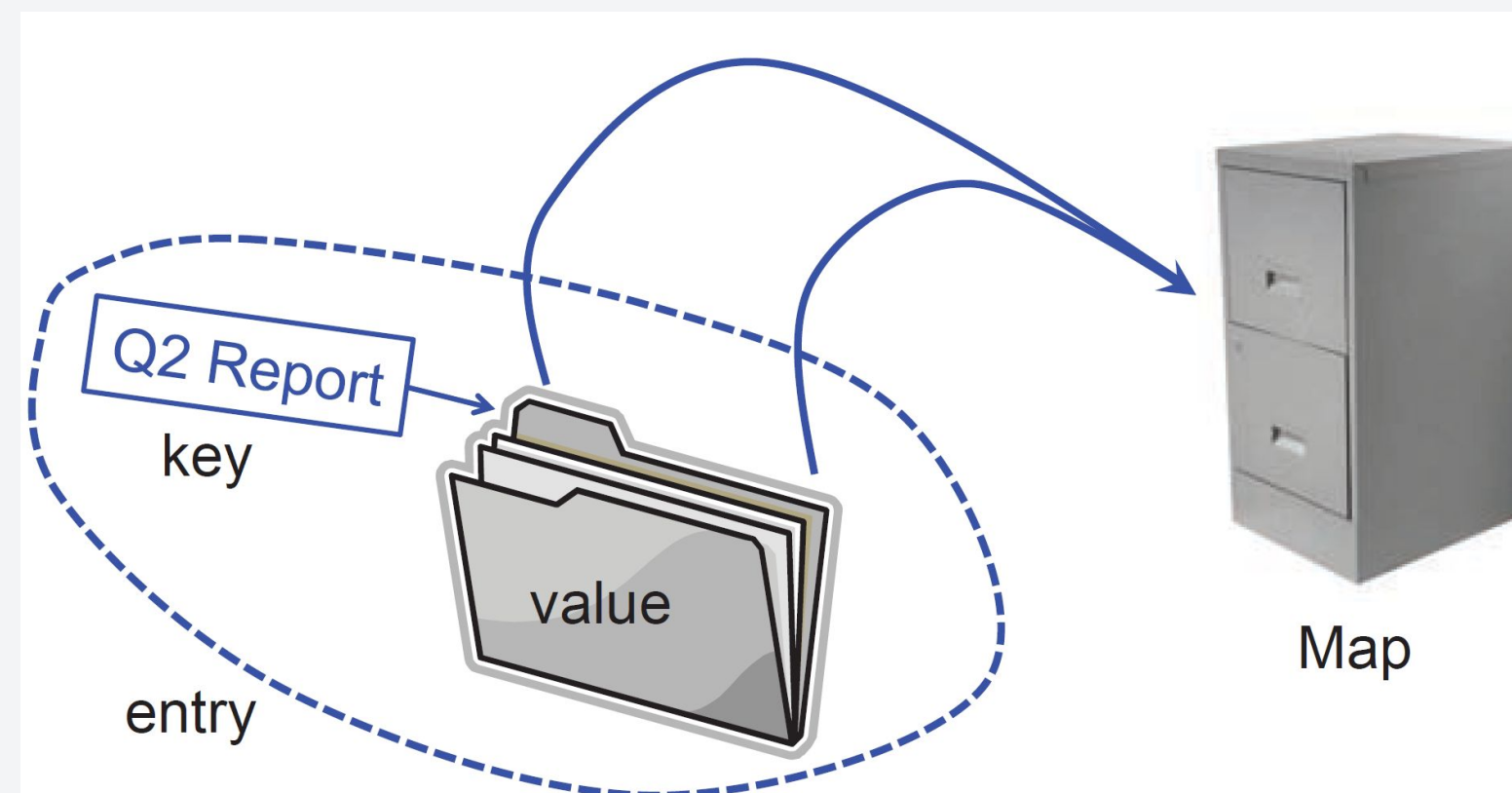
Insertion and Deletion of Data takes $O(\log n)$ complexity. Verify!!!

- Trees and Heap
- **Hash Map**

Maps

Hash maps uses the concept of **key-value** pairs.

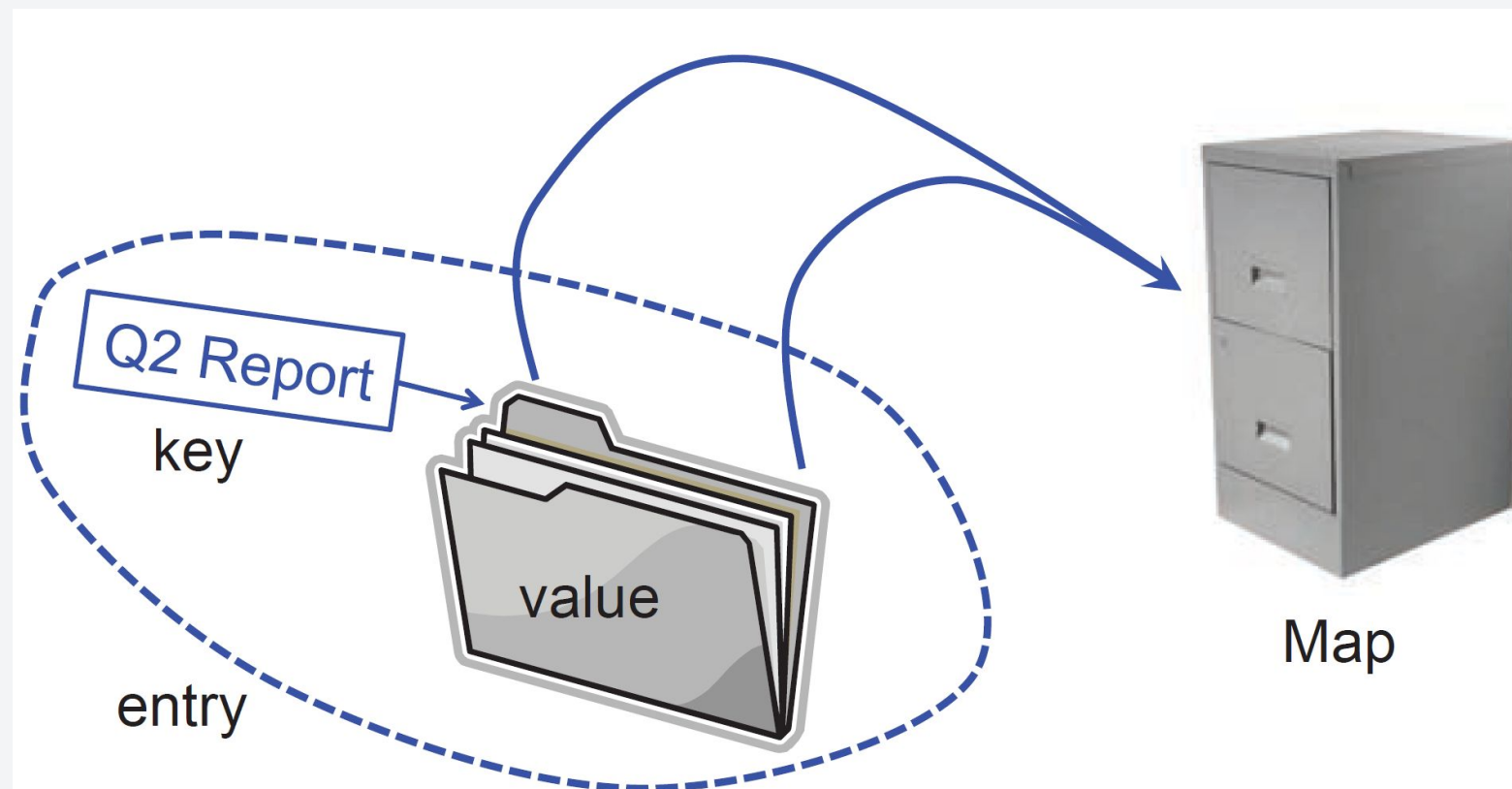
- Each element is assigned a **key** using a hash function.
- Each key will then be mapped to certain elements, which we consider as the **value**.
- The hash map stores this key-value pairs.



Maps

Hash maps uses the concept of **key-value** pairs.

- Each element is assigned a **key** using a hash function.
- Each key will then be mapped to certain elements, which we consider as the **value**.
- The hash map stores this key-value pairs.



Example:

- The contents of the document are considered the **value**.
- Each document are labelled with a name considered as the **key**.
- They are then stored in a file cabinet.

Properties of Maps

Stores data using key-value pairs (k,v) called **entries**.

- Keys should be unique.
- Keys can be any type as long as they have well-defined relational operators (why?)

To access and store data, we need to have the specific key.

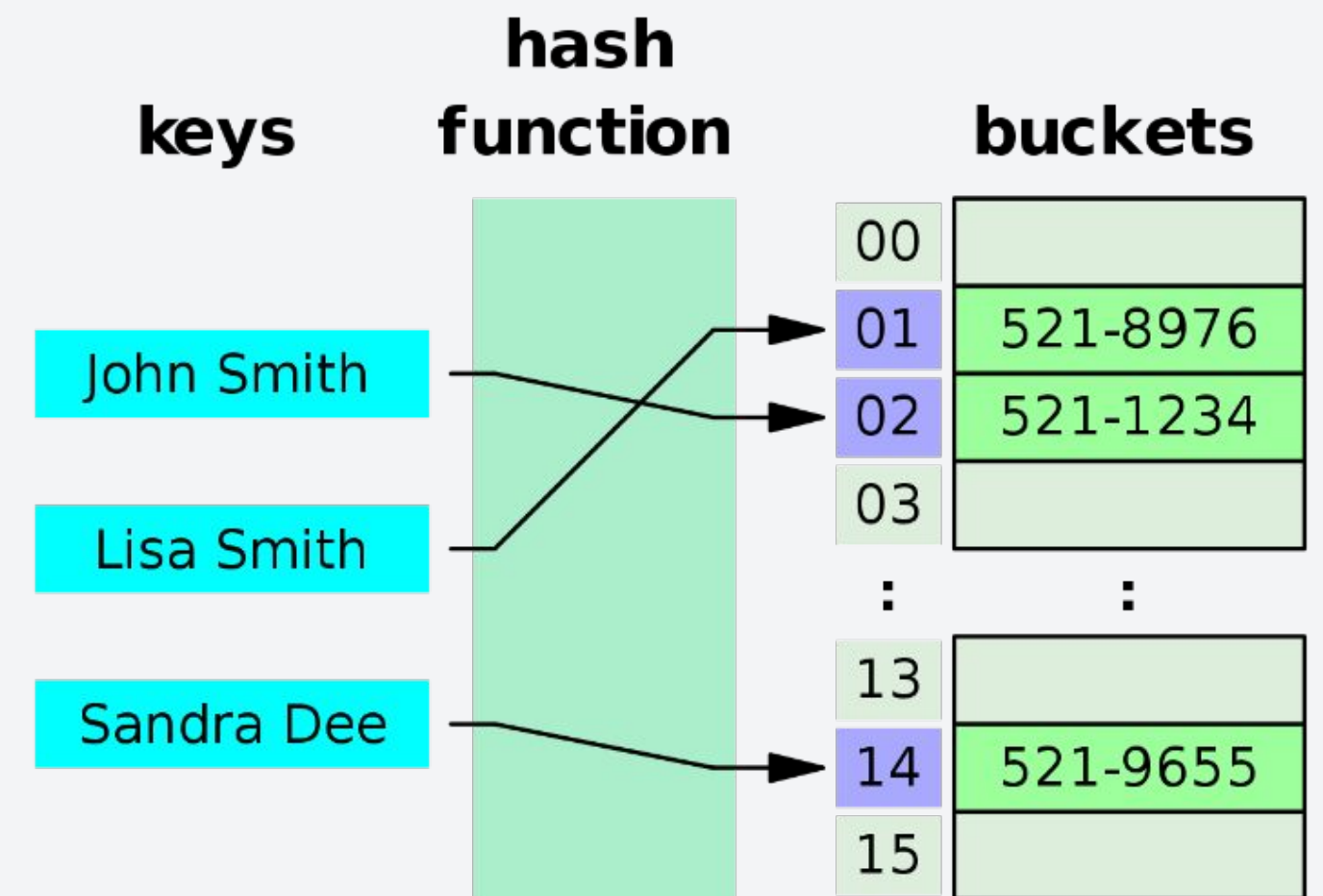
- Insertion an element: Generate a key, store the element based on the key.
- Deleting an element: Specify the key, delete the element that has the corresponding key.

The most known implementation of maps is using **Hash Tables**.

Hash Table Basics

A hash table is a non-linear data structure that stores information based on a hashing rule or hash function. It is composed of two components:

1. **Hash Function:** a function that converts any data (in this case the key) into a simplified reference to the data (in this case to an integer)
2. **Bucket Array:** this serves as the container that holds the data of each key-value pair. Each bucket can usually store more than one data (why? we'll see later)



Let's practice hashing

Suppose we want to convert a word to some integer value using a hash function.

Consider a hash function that:

- Converts each letter to its placement on the alphabet (a - 1, b - 2, ...) and converts the decimal number to five-digit binary.
- We then concatenate all letters and transform it to a 32-bit number.

Let's practice hashing

Suppose we want to convert a word to some integer value using a hash function.

Consider a hash function that:

- Converts each letter to its placement on the alphabet (a - 1, b - 2, ...) and converts the decimal number to five-digit binary.
- We then concatenate all letters and transform it to a 32-bit number.

Example: What is the hash value of the word “code”

Let's practice hashing

Suppose we want to convert a word to some integer value using a hash function.

Consider a hash function that:

- Converts each letter to its placement on the alphabet (a - 1, b - 2, ...) and converts the decimal number to five-digit binary.
- We then concatenate all letters and transform it to a 32-bit number.

Example: What is the hash value of the word “code”

1. Convert to decimal, then

binary

$$c = 3_{10} = 00011_2$$

$$o = 15_{10} = 01111_2$$

$$d = 4_{10} = 00100_2$$

$$e = 5_{10} = 00101_2$$

Let's practice hashing

Suppose we want to convert a word to some integer value using a hash function.

Consider a hash function that:

- Converts each letter to its placement on the alphabet (a - 1, b - 2, ...) and converts the decimal number to five-digit binary.
- We then concatenate all letters and transform it to a 32-bit number.

Example: What is the hash value of the word “code”

1. Convert to decimal, then 2. Concatenate

binary

$$c = 3_{10} = 00011_2$$

$$o = 15_{10} = 01111_2$$

$$d = 4_{10} = 00100_2$$

$$e = 5_{10} = 00101_2$$

$$\text{code} = 00011\ 01111\ 00100\ 00101_2$$

Let's practice hashing

Suppose we want to convert a word to some integer value using a hash function.

Consider a hash function that:

- Converts each letter to its placement on the alphabet (a - 1, b - 2, ...) and converts the decimal number to five-digit binary.
- We then concatenate all letters and transform it to a 32-bit number.

Example: What is the hash value of the word “code”

1. Convert to decimal, then 2. Concatenate

binary

$$c = 3_{10} = 00011_2$$
$$o = 15_{10} = 01111_2$$
$$d = 4_{10} = 00100_2$$
$$e = 5_{10} = 00101_2$$

$$\text{code} = 00011\ 01111\ 00100\ 00101_2$$

3. Transform to 32-bits

$$\text{code} = 00\ 00000\ 00000\ 00011\ 01111\ 00100\ 00101_2$$

Let's practice hashing

Suppose we want to convert a word to some integer value using a hash function.

Consider a hash function that:

- Converts each letter to its placement on the alphabet (a - 1, b - 2, ...) and converts the decimal number to five-digit binary.
- We then concatenate all letters and transform it to a 32-bit number.

Example: What is the hash value of the word “code”

1. Convert to decimal, then
2. Concatenate

binary

$$c = 3_{10} = 00011_2$$
$$o = 15_{10} = 01111_2$$
$$d = 4_{10} = 00100_2$$
$$e = 5_{10} = 00101_2$$

$$\text{code} = 00011\ 01111\ 00100\ 00101_2$$

3. Transform to 32-bits

$$\text{code} = 00\ 00000\ 00000\ 00011\ 01111\ 00100\ 00101_2$$

Can you identify any potential issue?

Let's practice hashing

If we try to hash a word that is longer 6 letters, we have to remove some of the bits since we can only handle 32-bits. This limits the word length.

- Example: 'picking' = **10000 01001 00011 01011 01001 01110 00111**


This will be the final 32-bit result of the hash and the rest will be discarded

Let's practice hashing

If we try to hash a word that is longer 6 letters, we have to remove some of the bits since we can only handle 32-bits. This limits the word length.

- Example: 'picking' = **10000 01001 00011 01011 01001 01110 00111**



This will be the final 32-bit result of the hash and the rest will be discarded

Additionally, some words can have the same hash resulting into **collisions**.

- Example: 'picking' = **10000 01001 00011 01011 01001 01110 00111**
- Example: 'ticking' = **10100 01001 00011 01011 01001 01110 00111**

Let's practice hashing

If we try to hash a word that is longer 6 letters, we have to remove some of the bits since we can only handle 32-bits. This limits the word length.

- Example: 'picking' = **10000 01001 00011 01011 01001 01110 00111**


This will be the final 32-bit result of the hash and the rest will be discarded

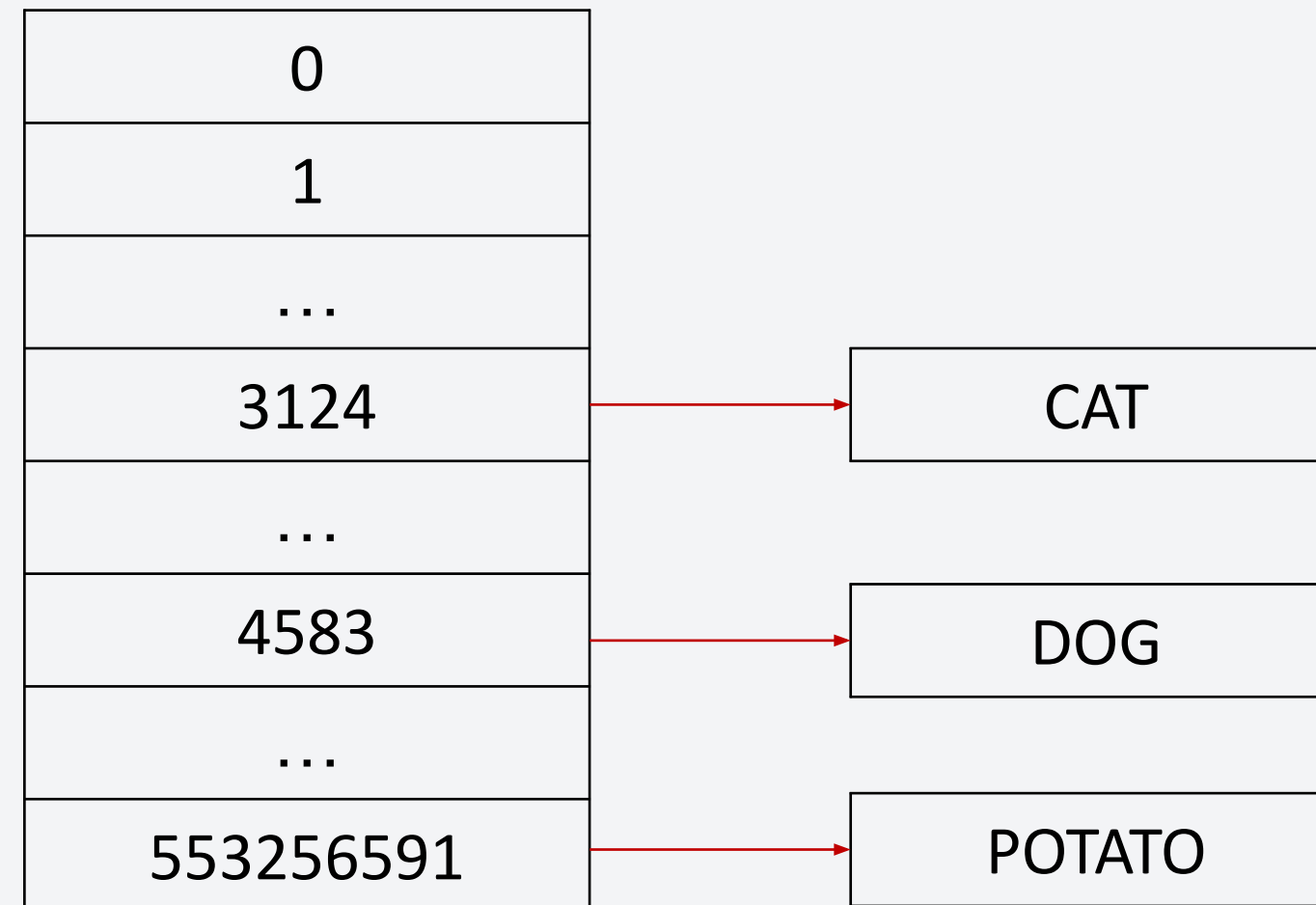
Additionally, some words can have the same hash resulting into **collisions**.

- Example: 'picking' = **10000 01001 00011 01011 01001 01110 00111**
- Example: 'ticking' = **10100 01001 00011 01011 01001 01110 00111**

The first problem can be solved by choosing a better hashing function. But the second problem will still appear even if we change the hashing function.

How do we handle collisions?

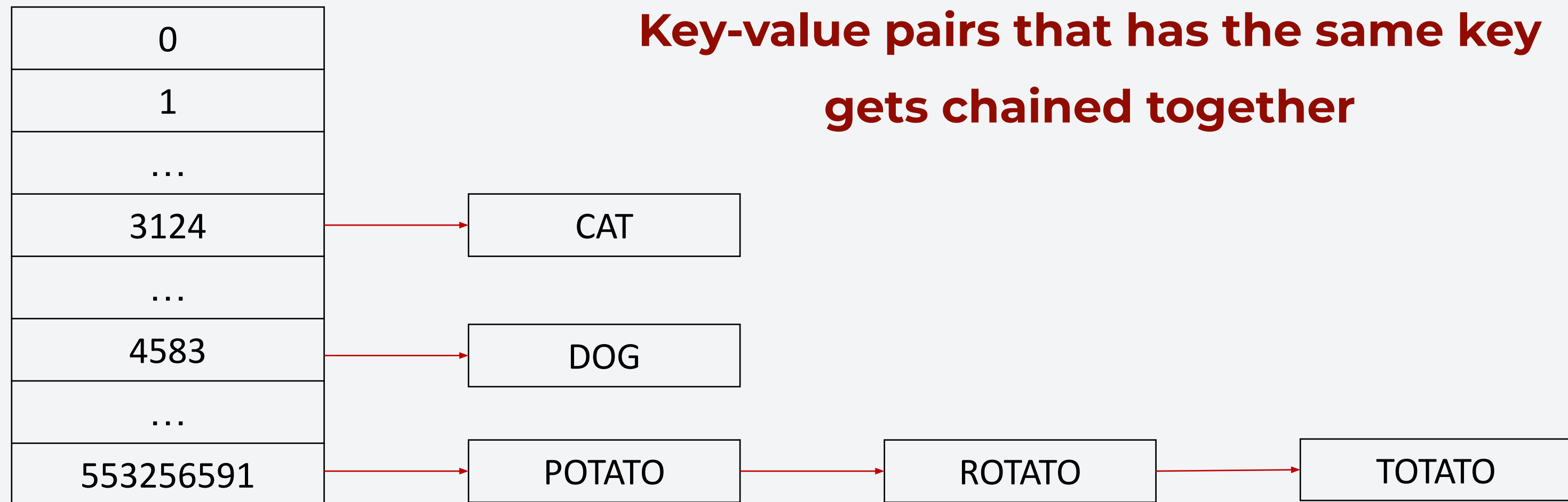
The most intuitive way is to increase the number of possible values for our hash function i.e. the result can be from 1 to 10^6 , but this results in a large number buckets.



How do we handle collisions?

The other method that we can do to address collisions is by using **bucket chaining**.

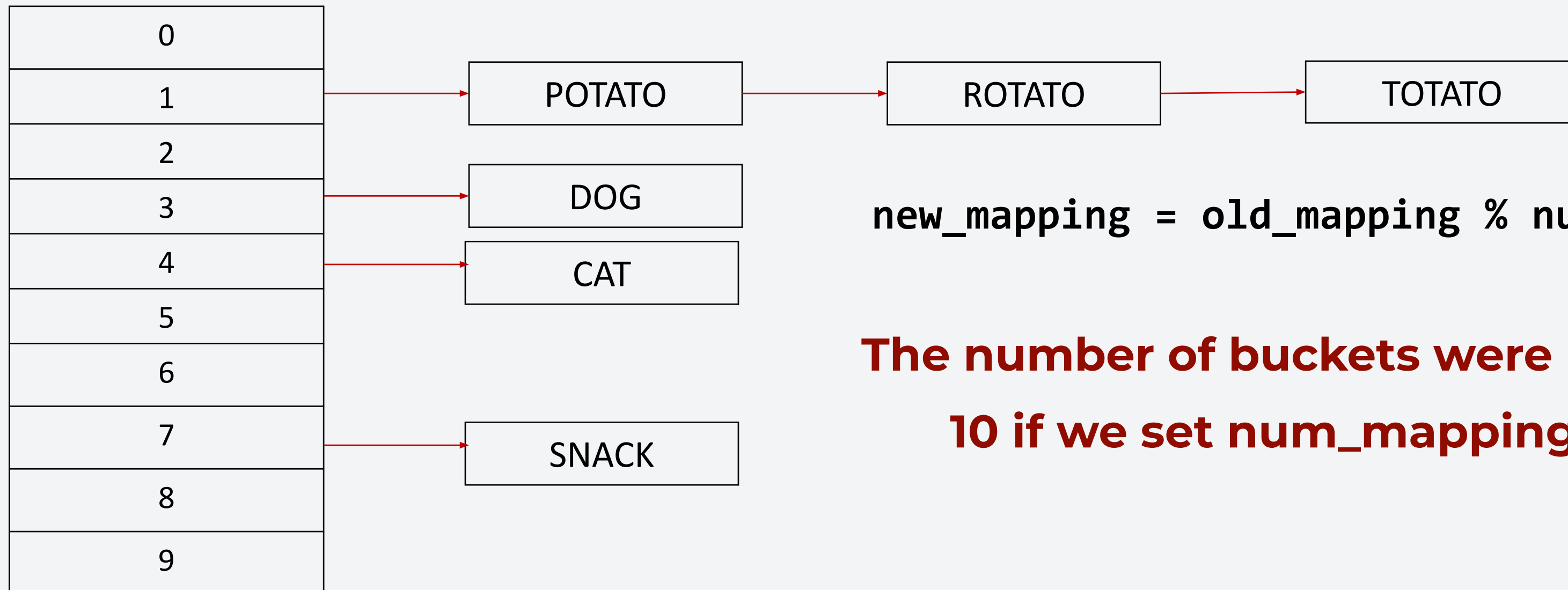
- Each bucket contains a secondary data structure (can be a tree or a linked list)



How do we handle collisions?

The other method that we can do to address collisions is by using **bucket chaining**.

- Each bucket contains a secondary data structure (can be a tree or a linked list)
- To minimize the number of buckets, we usually employ a modulo operation.



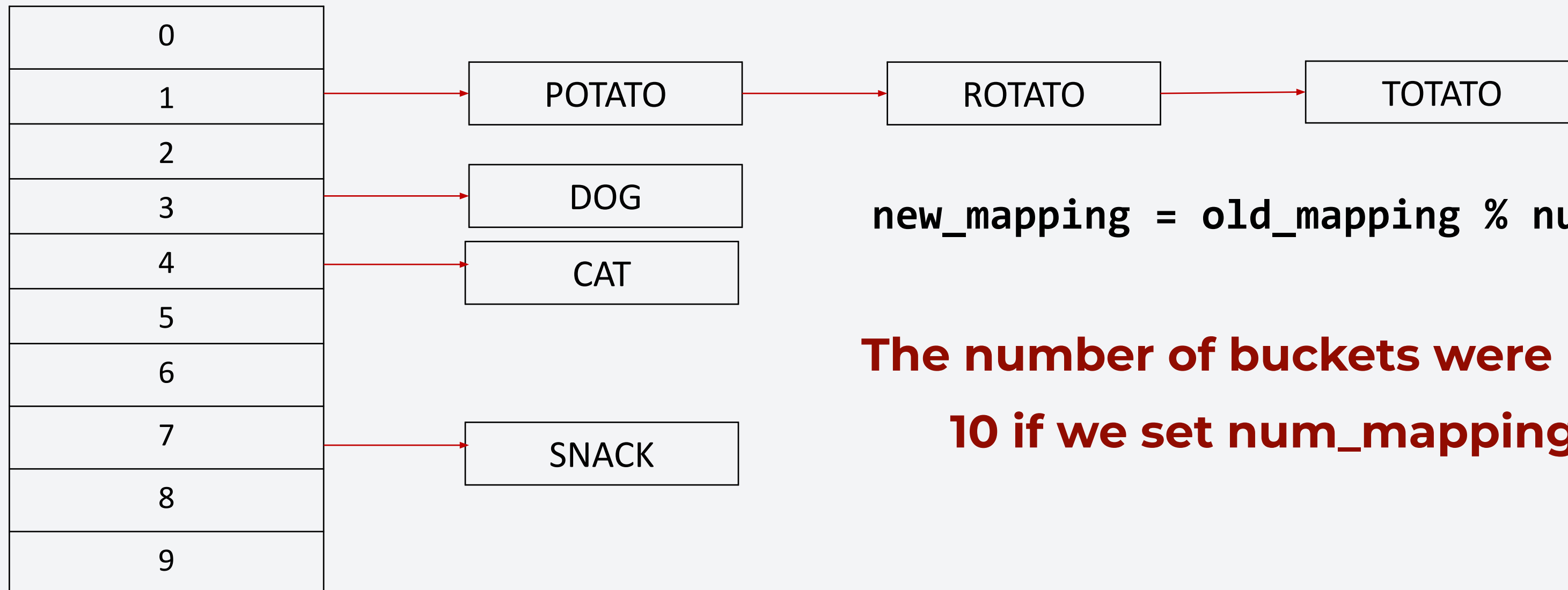
$$\text{new_mapping} = \text{old_mapping} \% \text{num_mappings}$$

The number of buckets were reduced to 10 if we set num_mappings to 10.

How do we handle collisions?

The other method that we can do to address collisions is by using **bucket chaining**.

- Each bucket contains a secondary data structure (can be a tree or a linked list)
- To minimize the number of buckets, we usually employ a modulo operation.



$\text{new_mapping} = \text{old_mapping} \% \text{num_mappings}$

The number of buckets were reduced to 10 if we set num_mappings to 10.

How to decide the bucket count?

Let,

- N be the number of items distributed along the hash table
- M be the number of buckets
- Q be the number of items in the longest chain

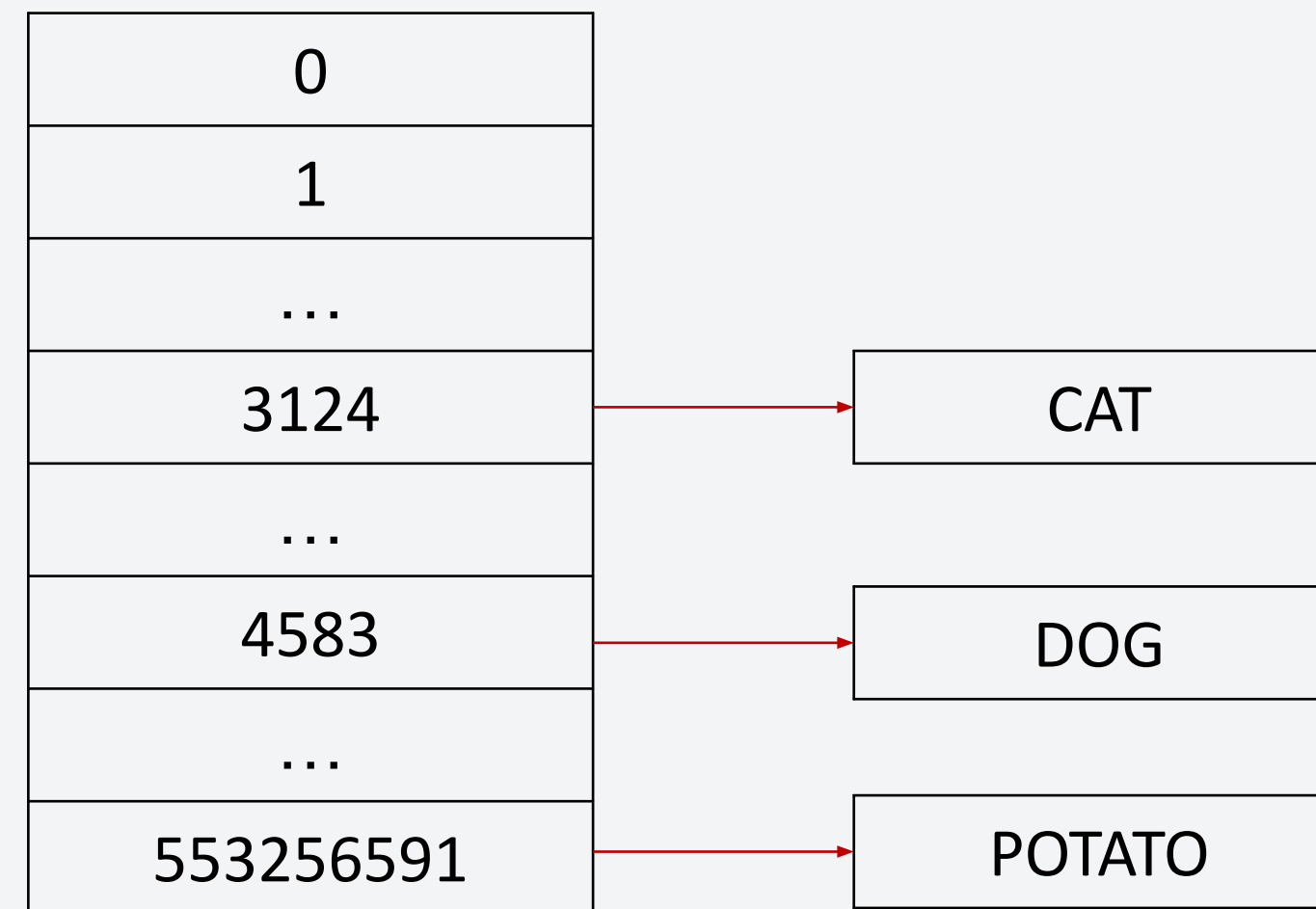
The load factor, L , is computed as $L = N/M$.

- The average runtime of inserting, deleting an element is $O(L)$ while the worst case runtime is $O(Q)$.
- Smaller M leads to higher Q . Thus, average and worst case runtime is worst.
- Increasing M leads to better time complexity but requires more memory.
- **There is a tradeoff!**

How to decide the bucket count?

Example:

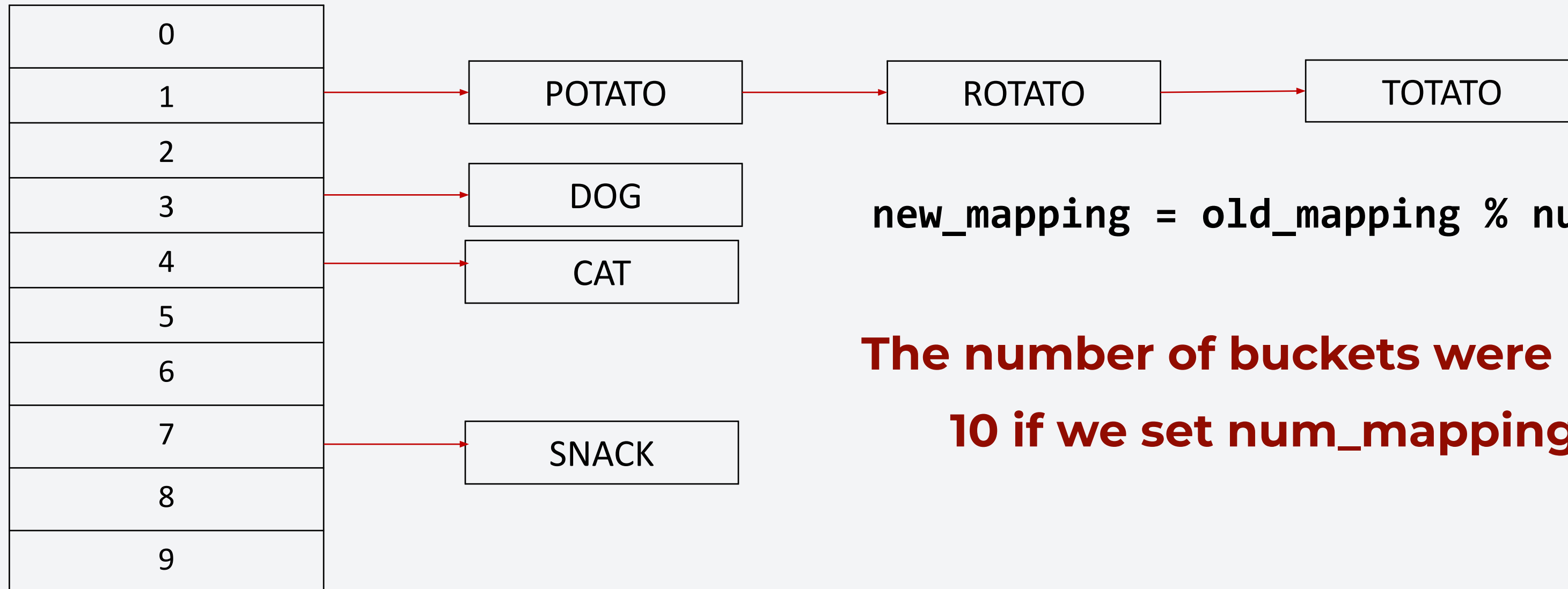
- Large bucket size (M)
- Less chaining (small Q), better worst case but more memory needed



How to decide the bucket count?

Example:

- Small bucket size (M)
- More chaining (large Q), worst worst case, less memory requirement



$\text{new_mapping} = \text{old_mapping} \% \text{num_mappings}$

The number of buckets were reduced to 10 if we set num_mappings to 10.

HashMap Libraries

C++ already has a built-in map structure called **STL Map**.

Usage:

```
#include <iostream>
#include <utility>
#include <map>
#include <string>
using namespace std;

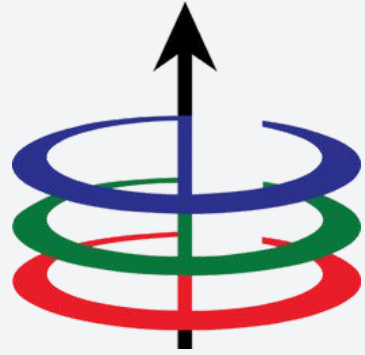
int main() {
    map<string,int> m; // initialization: map<key_type, value_type> map_variable;
    map<string,int>::iterator it;
    m.insert(make_pair("hello",9));
    m["world"] = 5;
    cout << m["world"] << endl; //output 5
    it = m.find("hello");
    m.erase(it); //remove key-value pair with key="hello"
    return 0;
}
```

Demo: Hash

Any questions?

Wala na? Quiz tayó!

Joke lang! Activity nalang :D



Lecture 5

Non-linear Data Structures

EEE 121 2s2526

Steven S. Sison